



**COLLEGE OF INFORMATION TECHNOLOGY AND
COMPUTER SCIENCE**

**GUBAT: DEVELOPMENT OF A RANDOM FOREST-
BASED SMS SPAM FILTER USING LOCALIZED DATASET**

In Partial Fulfillment of
CSCN13C-Software Engineering 2

Submitted by:

AlZarouni, Yoshida G.
Torres, Dominic
Valzado, Aeron C.
BSCS | 3rd Yr | CS 302

Date Submitted:

May 22, 2026

SOFTWARE REQUIREMENTS SPECIFICATION

CSCN13C-Software Engineering 2

1. Introduction and Background

SMS has become increasingly weaponized by cybercriminals. Recent data reveals an alarming surge in SMS-based fraud, with the Philippines recording approximately 6.1 million SMS scams and 610,000 scam calls in 2024 alone (InsiderPH, 2024). The impact on Filipino consumers is profound, with 41 percent of respondents reporting being targeted by smishing attacks in 2024 (Balita, 2024). The Philippines is among the most vulnerable nations globally, with a suspected digital fraud rate of 13.4% in 2024, ranking second highest in the world after India (ScamWatchHQ, 2025).

Mobile users are frequently targeted by phishing attempts, malicious links, and unsolicited promotional messages (National Telecommunications Commission [NTC], 2022). According to (Abid et al. 2022), Random Forest, when paired with TF-IDF and bag-of-words, can achieve up to 99% accuracy in distinguishing spam from legitimate messages. However, existing datasets used in such research are typically based on English SMS.

To bridge all this gap, this project aims to develop a Random Forest-based SMS spam filter trained on a localized Filipino-English dataset and integrated into a mobile application.

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to provide a comprehensive and detailed description of the requirements for the Random Forest-Based SMS Spam Filter system and its accompanying mobile application. This document serves as the formal specification for the continuation of development under Software Engineering 2, building upon the foundational work established in Software Engineering 1.

This SRS defines the functional and non-functional requirements of the system, establishes the basis for validation and verification, and serves as a binding agreement between the development team and all stakeholders. It is intended to guide the complete development lifecycle, from design through testing and deployment.

1.2 Intended Audience and Reading Suggestions

The intended audience for this System Requirements Specification includes:

Developers – Software engineers and mobile app developers responsible for implementing the SMS spam filter system and the mobile application. Developers should read all sections in full, with particular focus on Section 3 (System Features), Section 4 (External Interface Requirements), Section 5 (Non-Functional Requirements), and Section 7 (System Implementation Requirements). These sections define the technical scope and constraints of the system.

SMS Users/End Users – Everyday Filipino mobile phone users who will use the application to manage and filter spam messages. End users should review Section 1.3 (Product Scope), Section 2.2 (Product Functions), and Section 4.1 (User Interfaces) for an overview of what the system does and how it can be used.

Testers – Quality assurance professionals responsible for testing and validating the system. Testers should focus on Section 3 (System Features and Functional Requirements), Section 5 (Non-Functional Requirements), and Section 8 (System Testing Requirements), which define the expected behaviors, performance benchmarks, and acceptance criteria used for test design and validation.

Suggested Reading Order:

- For Developers:
 - Section 3 (System Features)
 - Section 4 (External Interface Requirements)
 - Section 5 (Non-Functional Requirements)
 - Section 7 (System Implementation Requirements)
- For SMS Users/End Users:
 - Section 1.3 (Product Scope)
 - Section 2.2 (Product Functions)
 - Section 4.1 (User Interfaces)
 - Appendix B (Screenshots)
- For Testers:
 - Section 3 (System Features and Functional Requirements)
 - Section 5 (Non-Functional Requirements)
 - Section 8 (System Testing Requirements)

1.3 Product Scope of the System

The system is a Random Forest-based SMS spam filtering solution tailored for Filipino mobile phone users. It consists of two integrated components: a Flutter-based Android mobile application and a backend classification API called GubatDetectAPI.

The Philippines is among the most digitally vulnerable nations globally, with approximately 6.1 million SMS scams recorded in 2024 alone. Existing spam filtering solutions fail to address localized spam patterns, mixed Filipino-English (Taglish) messages, and obfuscated text common in Philippine SMS fraud. This system aims to bridge that gap.

The mobile application provides users with real-time spam detection, inbox and spam folder management, manual override capabilities, sender blocking, message search, and optional spam reporting. The classification engine uses a Random Forest model trained on a localized Filipino-English dataset, achieving a minimum target accuracy of 90%.

The system supports only Android devices running version 13 or above and processes SMS text messages only, MMS and phone calls are outside scope. On-device

model deployment is not supported due to limited TensorFlow Lite compatibility on Android; classification is handled server-side via GubatDetectAPI.

1.4 Main Problem

SMS spam is a constant and evolving issue affecting Filipino mobile users, risking security and privacy. Existing spam filtering techniques are ineffective in the local context, failing to detect and filter unique spam patterns, mixed Filipino-English messages, and obfuscation of words. As a consequence, users are exposed to targeted texts, unwanted links, phishing attempts from scam artists, spent time manually deleting spam, and a decreased reliance on SMS as a dependable means of communication.

The primary issue is the lack of an adaptive, customized SMS spam filtering system specifically developed for Filipino mobile users that can effectively detect between spam and legitimate messages.

1.4.1 Specific Problem

The main problem indicates three (3) specific problems:

1. There is an inadequate detection of localized SMS spam patterns.
2. The continuous evolving SMS spam tactics to evade detection.
3. There is an insufficient assessment of localized SMS spam patterns.

1.5 General Objectives

To develop an effective, localized SMS spam filtering system utilizing the Random Forest machine learning algorithm to accurately classify Filipino-English SMS messages as spam or legitimate (ham), as well as to provide Filipino mobile users with an Android application for managing SMS messages that is seamlessly integrated with the SMS spam filtering system. Subsequently, an assessment of SMS spam features can be concluded from this project, that may be beneficial to future developers.

1.5.1 Specific Objectives

To achieve the general objective, it aims to accomplish these specific objectives:

1. To develop a functional Android mobile application integrating SMS filtering system with a Random Forest machine learning algorithm at its core.
2. To compile a localized dataset containing Filipino-English SMS messages.
3. To identify and rank the most distinctive characteristics of Filipino-English SMS spam.

2. Overall Description

2.1 Product Perspective

The SMS spam filtering system and its mobile app are designed to provide a secure and streamlined way to manage incoming text messages, effectively shielding them from scams, phishing attempts, and unsolicited spam. This is particularly valuable for Filipino mobile phone users, where scam SMS messages have become a pervasive issue. By integrating tailored filtering technology into an everyday messaging tool, the product aims to reduce digital stress and promote safer communication.

The system is not a standalone product. It operates within a client-server architecture composed of three integrated components: the Flutter-based Android mobile application (client), the GubatDetectAPI (server), and Google Firebase Firestore (cloud database). The mobile application handles all user-facing operations and local data storage, while classification requests are offloaded to GubatDetectAPI over HTTPS. Spam reports submitted by users are stored remotely in Firebase Firestore. The mobile app interacts with two external systems: GubatDetectAPI for real-time SMS classification, and Google Firebase Firestore for anonymous spam report collection.

2.2 Product Functions

The core product is a simplified Android messages app designed to demonstrate the SMS spam filtering system. It prioritizes simplicity, ease of use, and automatic operation to protect users from scams and unwanted messages.

Key features include:

- Automatic Message Classification – The app receives incoming SMS messages and uses the trained random forest model to automatically classify them as spam or ham in real-time.
- Spam Filtering – Detected spam messages are filtered out and moved to a dedicated spam folder, preventing them from cluttering the main inbox and reducing exposure to potential scams.
- Inbox Management – Ham messages remain in the inbox, ensuring users have immediate access to important communications.
- Manual Overrides – Users can manually mark messages as spam or unmark them if incorrectly classified.
- Search Function – Users can perform searches across all conversations in the inbox or spam folder using keywords or sender names. Additionally, search function also works inside an individual conversation thread.
- Sender/Number Blocking – Users can block specific senders or phone numbers, preventing future messages from those sources from appearing in the inbox and going directly to the spam folder, with options to manage blocked numbers directly within the app.
- Optional Spam Reporting – Users have the option to report confirmed spam messages directly to the developers anonymously.

2.3 User Classes and Characteristics

User Class	Description	Skill Level	Responsibilities
SMS Users / End Users	Everyday Filipino mobile phone users who receive frequent SMS messages and are the primary beneficiaries of the spam filtering system.	Low-Medium	They interact with the mobile application to manage their inbox, review spam, block senders, and optionally report spam. No technical background is required. They have access to all user-facing features of the mobile

			app including inbox, spam folder, search, blocking, and settings.
Developers	The development team responsible for building, maintaining, and retraining the system.	High	Developers have full access to the source code, GubatDetectAPI, the trained Random Forest model, Firebase Firestore, and the Docker deployment environment. They are expected to have proficiency in Flutter, Python, machine learning (YDF/scikit-learn), REST API development (FastAPI), and cloud services (Firebase, Docker, ngrok).

2.4 Operating Environment

Hardware Requirements - The system is designed exclusively for Android-based mobile phone devices running Android OS version 13 or higher. An active and registered SIM card must be present on the device to receive SMS messages. A stable internet connection with a minimum of 1 Mbps download/upload speed is required for API communication and cloud database access.

Software Requirements - The mobile application is built using Flutter and requires the Android SDK. The backend classification service (GubatDetectAPI) is developed in Python using the FastAPI framework, containerized via Docker, and exposed through an ngrok secure tunnel. Cloud-based spam report storage is handled via Google Firebase Firestore. The Random Forest model is implemented using the Yggdrasil Decision Forests (YDF) library. In addition, these flutter dependencies were used in developing the mobile application; another_telephony, hive & hive_flutter, firebase_core & cloud_firestore, permission_handler, http, cryptography_plus & cryptography_flutter_plus, flutter_dotenv, path_provider, fast_contacts, and flutter_local_notifications.

Operating Systems - The client-side application is compatible with Android 13 and above. The server-side API runs on a Linux-based Docker container environment.

2.5 Design and Implementation Constraints

The following are the design and implementation constraints of the system.

- The system is constrained to Android-only deployment; iOS is not supported.
- The Random Forest model cannot be deployed locally on-device due to limited TensorFlow Lite support for the YDF model format on Android. All inference must be performed server-side via GubatDetectAPI.

- The application requires Android permission grants for SMS access, Contacts access, and Notifications access in order to function. Core functionality is unavailable if these permissions are denied.
- The system is trained on Filipino-English (Taglish) SMS data and is optimized for that language context. Performance on purely foreign-language messages may be reduced.
- The spam reporting feature depends on internet connectivity. Reports are cached locally when offline and transmitted once connectivity is restored.
- The GubatDetectAPI is exposed through ngrok, which introduces dependency on ngrok service availability and may require URL updates upon redeployment.
- All SMS data transmitted to the API must be AES-256-GCM encrypted to comply with the system's security requirements.

2.6 User Documentation

The following user documentation is planned for the system:

A demo video is available to assist users in understanding core features, including how to mark or unmark messages as spam, how to block or unblock senders, and how to use the search function.

Developer documentation will be maintained as inline code comments and a README.md file within the project repository, covering API endpoints, model retraining procedures, and deployment steps via Docker.

No formal training materials are currently planned beyond the above, as the application is designed to require no more than three steps to complete any core function.

2.7 Assumptions and Dependencies

The mobile app and its SMS spam filtering system are built on foundational assumptions and dependencies about operational requirements.

The system assumes that the user will use an Android device running version 13 or higher. Users will grant necessary system permissions, including SMS Access, Contacts Access, and Notifications Access. An active and registered SIM card is available on the device.

The system is dependent on Android SDK, Flutter and associated libraries to support app development. A stable internet connection for accessing GubatDetectAPI and cloud database.

3. System Features (Functional Requirements)

This section provides all the features of the mobile app, including the description, stimuli/response sequences, and functional requirements of each feature.

System Feature – Mark as Spam & Move to Spam Folder
Description

Description:	This feature allows users to manually mark misclassified messages as spam and automatically move them to the spam folder.
Priority:	High
Stimulus/Response Sequences	
Stimulus 1:	User identifies a spam message in the Inbox and selects the message(s).
Response:	System displays the message(s) as selected with visual indication.
Stimulus 2:	User clicks the "Mark as Spam" option from the menu.
Response:	System displays a confirmation dialog box with: Confirmation message: "Mark this message(s) as spam and move to spam folder?", Checkbox option: "Report this as spam?", "Yes" and "Cancel" buttons.
Stimulus 3:	User left uncheck the "Report this as spam?" checkbox and clicks "Yes".
Response:	System marks the message(s) as spam, moves them to the Spam folder, displays confirmation message, and updates the Inbox.
Stimulus 4:	User clicks "Cancel" button in the confirmation dialog.
Response:	System closes the dialog without making any changes.
Functional Requirements	
Req. No.	Requirement
REQ-1	The system must allow the user to select one or multiple messages.
REQ-2	The system must display a confirmation dialog box when the user attempts to mark a message as spam.
REQ-3	The system must provide a checkbox option to optionally report as spam in the confirmation dialog.
REQ-4	The system must move selected message(s) from the Inbox to the Spam folder upon confirmation.
REQ-5	The system must send message data to developers when the checkbox to report as spam is checked.
REQ-6	The system must display a confirmation message after the operation completes.
REQ-7	The system must update the Inbox automatically after moving messages to Spam folder.
REQ-8	The system must allow the user to cancel the operation without making any changes.

System Feature – Unmark as Spam & Restore to Inbox	
Description	
Description:	This feature allows users to restore ham messages that were misclassified as spam back to the Inbox.
Priority:	High
Stimulus/Response Sequences	
Stimulus 1:	User identifies a ham message in the Spam folder and selects the message(s).
Response:	System displays the message(s) as selected with visual indication.
Stimulus 2:	User clicks the "Unmark as Spam" option from the menu.
Response:	System displays a confirmation dialog box with: Confirmation message: "Unmark this message as spam and restore to inbox?", "Yes" and "Cancel" buttons.
Stimulus 3:	User clicks "Yes" button in the confirmation dialog.
Response:	System unmarks the message(s) as spam, moves them to the Inbox, displays confirmation message, and updates the Spam folder.
Stimulus 4:	User clicks "Cancel" button in the confirmation dialog.
Response:	System closes the dialog without making any changes.
Functional Requirements	
Req. No.	Requirement
REQ-1	The system must allow the user to select one or multiple messages.
REQ-2	The system must display a confirmation dialog box when the user attempts to unmark a message as spam.
REQ-3	The system must move selected message(s) from the Spam folder to the Inbox upon confirmation.
REQ-4	The system must display a confirmation message after the operation completes.
REQ-5	The system must update the Spam folder automatically after moving messages to Inbox.
REQ-6	The system must allow the user to cancel the operation without making any changes.
System Feature – Report Message as Spam	
Description	

Description:	This feature allows users to report spam messages to developers to improve the spam filtering system's accuracy and effectiveness.
Priority:	Low
Stimulus/Response Sequences	
Stimulus 1:	User identifies a spam message in the Inbox and selects the message(s).
Response:	System displays the message(s) as selected with visual indication.
Stimulus 2:	User clicks the "Mark as Spam" option from the menu.
Response:	System displays a confirmation dialog box with: Confirmation message: "Mark this message(s) as spam and move to spam folder?", Checkbox option: "Report this as spam?", "Yes" and "Cancel" buttons.
Stimulus 3:	User checks the "Report this as spam?" checkbox and clicks "Yes" with internet connection.
Response:	System marks the message(s) as spam, moves them to the Spam folder, sends message data to developers, displays confirmation message, and updates the Inbox.
Stimulus 4:	User checks the "Report this as spam?" checkbox and clicks "Yes" with no internet connection.
Response:	System marks the message(s) as spam, moves them to the Spam folder, caches message data to local storage, displays confirmation message, updates the Inbox view, and sends cached data to developers once internet connection is restored.
Stimulus 5:	User clicks "Cancel" button in the confirmation dialog.
Response:	System closes the dialog without making any changes.
Functional Requirements	
Req. No.	Requirement
REQ-1	The system must allow the user to select one or multiple messages.
REQ-2	The system must display a confirmation dialog box when the user attempts to mark a message as spam.
REQ-3	The system must provide a checkbox option to optionally report as spam in the confirmation dialog.
REQ-4	The system must move selected message(s) from the Inbox to the Spam folder upon confirmation regardless of internet connection status.
REQ-5	The system must send message data to developers when the checkbox to report as spam is checked.

REQ-6	The system must display a confirmation message after the operation completes.
REQ-7	The system must update the Inbox automatically after moving messages to Spam folder.
REQ-8	The system must allow the user to cancel the operation without making any changes.
REQ-9	The system must cache message data to local storage when there is no internet connection.
REQ-10	The system must automatically send cached spam reports to developers once internet connection is restored.

System Feature – View Inbox	
Description	
Description:	This feature allows users to access and display all received messages that are not marked as spam.
Priority:	High
Stimulus/Response Sequences	
Stimulus 1:	User selects the Inbox icon option from the bottom navigation menu.
Response:	System retrieves all messages categorized as inbox from local storage.
Response:	Displays the inbox messages, including sender, last message, and timestamp.
Stimulus 2:	User selects a conversation to open.
Response:	System displays the full content of the selected conversation.
Functional Requirements	
Req. No.	Requirement
REQ-1	The system must display an Inbox icon in the bottom navigation menu.
REQ-2	The system must retrieve all ham messages from local storage when the Inbox is accessed.
REQ-3	The system must display a list of messages showing sender name/number, last message preview, and timestamp.
REQ-4	The system must allow users to select and open individual conversations.
REQ-5	The system must display the full content of messages of a conversation when selected.

REQ-6	The system must sort messages by timestamp with the most recent conversations appearing first.
REQ-7	The system must include both automatically filtered and manually marked ham messages.

System Feature – View Spam Folder	
Description	
Description:	This feature allows users to access and display all messages that have been automatically or manually classified as spam.
Priority:	High
Stimulus/Response Sequences	
Stimulus 1:	User selects the Spam Folder icon option from the bottom navigation menu.
Response:	System retrieves all messages categorized as spam from local storage.
Response:	Displays the spam messages, including sender, last message, and timestamp.
Stimulus 2:	User selects a conversation to open.
Response:	System displays the full content of the selected conversation.
Functional Requirements	
Req. No.	Requirement
REQ-1	The system must display a spam folder icon in the bottom navigation menu.
REQ-2	The system must retrieve all spam messages from local storage when the spam folder is accessed.
REQ-3	The system must display a list of messages showing sender name/number, last message preview, and timestamp.
REQ-4	The system must allow users to select and open individual conversations.
REQ-5	The system must display the full content of messages of a conversation when selected.
REQ-6	The system must sort messages by timestamp with the most recent conversations appearing first.
REQ-7	The system must include both automatically filtered and manually marked spam messages.

System Feature – Search Conversations	
Description	
Description:	This feature allows users to quickly locate specific messages or conversation threads within the Inbox or Spam folder.
Priority:	Medium
Stimulus/Response Sequences	
Stimulus 1:	User selects the Search icon option at the menu.
Response:	System displays a search input field.
Stimulus 2:	User enters a keyword, phrase, or sender name in the search field.
Response:	System scans all messages and sender names in the current folder.
Response:	System displays all conversations that match the search term.
Response:	If no matches are found, system displays the message "No messages found".
Functional Requirements	
Req. No.	Requirement
REQ-1	The system must display a search icon in the menu.
REQ-2	The system must display a search input field when the search icon is selected.
REQ-3	The system must scan all messages and sender names in the currently active folder.
REQ-4	The system must allow users to select and open individual conversations.
REQ-5	The system must support searching by keyword, phrase, or sender name.
REQ-6	The system must display all conversations that contain matches to the search term.
REQ-7	The system must display "No messages found" when no matches are found.

System Feature – Search Messages within a Conversation	
Description	
Description:	This feature allows users to find specific words or phrases inside a conversation thread.
Priority:	Medium

Stimulus/Response Sequences	
Stimulus 1:	User opens a specific conversation thread.
Response:	System displays the full conversation thread.
Stimulus 2:	User selects the Search icon option at the menu.
Response:	System displays a search input field.
Stimulus 3:	User enters a keyword, phrase, or sender name in the search field.
Response:	System scans all messages within the selected conversation.
Response:	System displays all messages that match the search term.
Response:	If no matches are found, system displays the message "No messages found".
Functional Requirements	
Req. No.	Requirement
REQ-1	The system must display a search icon in the menu.
REQ-2	The system must display a search input field when the search icon is selected.
REQ-3	The system must scan all messages and sender names in the currently active conversation thread.
REQ-4	The system must support searching by keyword, or phrase.
REQ-5	The system must display all messages that contain matches to the search term.
REQ-6	The system must display "No messages found" when no matches are found.

System Feature – Block Senders/Numbers	
Description	
Description:	This feature allows users to block unwanted contacts or phone numbers and go directly to spam folder.
Priority:	Medium
Stimulus/Response Sequences	
Stimulus 1:	User selects a conversation and chooses "Block" from the menu.
Response:	System adds the phone number to the app's blocklist, displays confirmation message, and automatically moves future messages from this number to Spam folder.

Stimulus 2:	User opens option menu and selects "Blocked Numbers".
Response:	System displays a list of all blocked numbers.
Stimulus 3:	User selects the "+" button to add a number manually.
Response:	System displays an input field requesting the phone number.
Stimulus 4:	User enters the phone number and selects "Block".
Response:	System adds the number to the blocked list and displays confirmation.
Functional Requirements	
Req. No.	Requirement
REQ-1	The system must allow users to block a sender directly from a main view through a menu option.
REQ-2	The system must maintain a blocklist of phone numbers in local storage (encrypted).
REQ-3	The system must add selected phone numbers to the blocklist upon confirmation.
REQ-4	The system must display a confirmation message upon successful operation.
REQ-5	The system must automatically move all future messages from blocked numbers to the Spam folder.
REQ-6	The system must display a list of all currently blocked numbers.
REQ-7	The system must allow users to manually add phone numbers to the blocklist.
REQ-8	The system must validate phone number format before adding to the blocklist.
REQ-9	The system must provide access to the "Blocked Numbers" list through the options menu.

System Feature – Unblock Senders/Numbers	
Description	
Description:	This feature allows users to unblock contacts or phone numbers from the blocked list.
Priority:	Medium
Stimulus/Response Sequences	
Stimulus 1:	User opens option menu and selects "Blocked Numbers".

Response:	System displays a list of all blocked numbers with a trash can icon next to each entry.
Stimulus 2:	User selects the trash can icon button next to a blocked number.
Response:	System removes the number from the blocked list and displays confirmation message.
Functional Requirements	
Req. No.	Requirement
REQ-1	The system must provide access to the "Blocked Numbers" list through the options menu.
REQ-2	The system must display all currently blocked numbers.
REQ-3	The system must remove the selected phone number from the blocklist when the trash icon is clicked.
REQ-4	The system must display a confirmation message upon successful operation.
REQ-5	The system must allow future messages from unblocked numbers to be received normally.
REQ-6	The system must display a list of all currently blocked numbers.
REQ-7	The system must update the blocked numbers list view immediately after unblocking.

System Feature – Test API Connection	
Description	
Description:	This feature allows users to verify whether the app can successfully communicate with the GubatDetectAPI.
Priority:	Low
Stimulus/Response Sequences	
Stimulus 1:	User opens the Settings section and selects "Test API Connection".
Response:	System sends a request to the GubatDetectAPI and waits for a response.
Response:	If the connection is successful, system displays "GubatDetectAPI is running (ms)".
Response:	If the connection fails or there is no internet connection, system displays "GubatDetectAPI connection failed".
Functional Requirements	

Req. No.	Requirement
REQ-1	The system must provide a "Test API Connection" option in the Settings section.
REQ-2	The system must send a GET request to the GubatDetectAPI when the option is selected.
REQ-3	The system must implement a timeout mechanism for API connection attempts.
REQ-4	The system must display a status message after the operation.

System Feature – Allow SMS Access

Description

Description: This feature requests and enables the app to read incoming messages and access SMS data.

Priority: High

Stimulus/Response Sequences

Stimulus 1: User opens the app and has not previously allowed necessary permissions.

Response: System displays a permission request.

Stimulus 2: User reviews the permission request and taps "Allow".

Response: System grants SMS read/write access and proceeds to next permission request.

Stimulus 3: User taps "Deny".

Response: System displays message indicating permission is required.

Functional Requirements

Req. No.	Requirement
REQ-1	The system must request permission when the app is launched and not previously allowed.
REQ-2	The system must grant access when the user approves the permission request.
REQ-3	The system must be able to read incoming messages from the device inbox after permission is granted.
REQ-4	The system must handle permission denial gracefully.

System Feature – Allow Contacts Access

Description	
Description:	This feature enables the app to display sender names and whitelist numbers from the phone's contacts.
Priority:	High
Stimulus/Response Sequences	
Stimulus 1:	User opens the app and has not previously allowed necessary permissions.
Response:	System displays a permission request.
Stimulus 2:	User reviews the permission request and taps "Allow".
Response:	System grants Contacts access and proceeds to next permission request.
Stimulus 3:	User taps "Deny".
Response:	System displays message indicating permission is required.
Functional Requirements	
Req. No.	Requirement
REQ-1	The system must request permission when the app is launched and not previously allowed.
REQ-2	The system must grant access when the user approves the permission request.
REQ-3	The system must be able to retrieve contact names after permission is granted.
REQ-4	The system must handle permission denial gracefully.

System Feature – Allow Notifications Access	
Description	
Description:	This feature enables the app to display notifications when new SMS messages are received.
Priority:	High
Stimulus/Response Sequences	
Stimulus 1:	User opens the app and has not previously allowed necessary permissions.
Response:	System displays a permission request.
Stimulus 2:	User reviews the permission request and taps "Allow".

Response:	System grants Notifications access and proceeds to next permission request.
Stimulus 3:	User taps "Deny".
Response:	System displays message indicating permission is required.
Functional Requirements	
Req. No.	Requirement
REQ-1	The system must request permission when the app is launched and not previously allowed.
REQ-2	The system must grant access when the user approves the permission request.
REQ-3	The system must be able to display notifications after permission is granted.
REQ-4	The system must handle permission denial gracefully.

3.4. Use Case Reference/Error Handling

The system handles the following error and edge cases across features:

Permission Denied - If the user denies SMS, Contacts, or Notifications access, the system displays an informative message indicating the permission is required and restricts the dependent functionality gracefully without crashing.

No Internet Connection - Features requiring API communication (spam classification, spam reporting) fall back to cached operation. Spam reports are stored locally and sent once connectivity is restored. The classification result may be delayed or unavailable offline.

API Connection Failure - If GubatDetectAPI is unreachable, the system displays "GubatDetectAPI connection failed" and does not classify the incoming message until the connection is restored.

Invalid Input (Block Number) - When a user manually enters a phone number to block, the system validates the format before adding it to the blocklist and displays an error if the format is invalid.

Empty Search Results: If no conversations or messages match a search query, the system displays "No messages found."

Confirmation Dialogs - All destructive or irreversible actions (mark as spam, unmark as spam, block sender) require user confirmation via a dialog box before executing, preventing accidental changes.

4. External Interface Requirements

4.1 User Interfaces

The following are external user interface components required for the system, refer to Appendix B for screenshots:

- Inbox and Spam Folder – A main screen displaying the filtered inbox for ham messages, with a separate tab for the spam folder to review filtered messages.
- Message Interaction Controls – Options to view, mark/unmark as spam, block/unblock senders, and search conversations or individual messages.
- Settings and Preferences – A dedicated settings screen accessible from the main menu, allowing users to manage blacklists, test API connections, and allow/revoke app permissions.
- Feedback and Reporting – Simple checkbox for optional anonymous spam reporting.

4.2 Hardware Interfaces

The following are external hardware components required for the system:

- Android Platform – The system is designed exclusively for Android-based mobile phone device.
- Operating System – The minimum OS version the system is designed for is Android version 13 or newer.
- SIM Card - An active and registered SIM card must be present to receive SMS messages.
- Internet Connection – Requires a stable internet connection, must maintain 1 Mbps download/upload over the internet.

4.3 Software Interfaces

The following are external software components and services required for the system:

- Google Firebase-Firestore – The system will use Firestore database to securely store and retrieve all the users' SMS spam reports.
- GubatDetectAPI – A secured API endpoint; this will be responsible for the following: Accepting encrypted SMS messages as input.
 - Processing the input via the data preprocessing and trained Random Forest model.
 - Returning a classification prediction as output.
 - The API will enforce HTTPS, HTTP Bearer key authentication, and encryption key to ensure data confidentiality and integrity.
- Local Storage – The system will use a local NoSQL database to persistently store the following:
 - User-defined list of blocked numbers.
 - Messages classified as spam.
 - Messages classified as ham.

4.4 Communications Interfaces

All communication between the mobile application and external services is conducted over HTTPS to ensure data integrity and encrypted transmission.

GubatDetectAPI Communication - The mobile app communicates with GubatDetectAPI via RESTful HTTP requests over HTTPS. The API enforces HTTP Bearer key authentication on all endpoints. SMS message content is AES-256-GCM encrypted before transmission and decrypted server-side prior to model inference. The API is hosted within a Docker container and exposed through an ngrok secure tunnel.

Firebase Firestore Communication - Spam report data is transmitted to Google Firebase Firestore using Firebase's native SDK over HTTPS. Only the SMS message body is transmitted, no sender identity or personal information is included in reports, ensuring anonymous submission.

Local Storage Communication - The mobile application uses a local NoSQL database (on-device) for storing inbox messages, spam messages, and the blocked numbers list. The blocked numbers list is stored in encrypted form. No network communication is required for local storage operations.

Data Transmission Format - Data exchanged with GubatDetectAPI is formatted as JSON over HTTPS REST. Encrypted SMS payloads are transmitted as AES-256-GCM ciphertext within the JSON body.

5. Non-Functional Requirements

The system utilizes five (5) non-functional requirements. This includes performance requirements, usability requirements, maintainability requirements, compatibility requirements, and security requirements.

5.1 Performance Requirements

The system must have a fast and reliable spam filter to minimize delays in message handling. Specifically, it should process and classify incoming SMS messages in real-time, filtering within 2 second per message. The trained random forest model must achieve an accuracy rate of at least 90% and an F1-Score within 85-95%.

5.2 Usability Requirements

The app must have an intuitive and user-friendly design to accommodate a wide range of users. The interface must be simple, with clear navigation elements such as tabs for the inbox, spam folder, and settings. Features like marking messages as spam, blocking senders, searching conversations, and reporting spam should be accessible via straightforward gestures and require no more than three (3) steps to complete.

5.3 Maintainability Requirements

The system must be easy to update, debug, and enhance. Developers must be able to retrain the random forest model with new data or new hyperparameters without disrupting the overall functionality. Specifically, the model must have a quarterly update (every 90 days) with new samples of at least 10% of current dataset, retrained model accuracy must follow the original performance requirements and should not fall below less than 2% of the original model accuracy.

5.4 Compatibility Requirements

The system must operate reliably across a wide range of Android devices. It should be compatible with Android operating system versions 13 and above. The app must maintain interoperability with GubatDetectAPI and Google Firebase Firestore for cloud-based data storage without conflicts.

5.5 Security Requirements

The system must ensure user privacy and data integrity, especially given the sensitive nature of SMS content involving personal or financial information. All stored data, including SMS messages and phone numbers, should be encrypted and protected from unauthorized access. The API connections must use HTTPS protocols, enforce HTTP Bearer key authentication, and encrypt message content during transit. Spam reports should remain anonymous and not contain any information from the sender apart from the SMS message body.

5.6 Software Quality Attributes

Reliability - The model must maintain consistent spam classification performance across diverse Filipino-English SMS inputs, including obfuscated text and code-switched messages, with no single feature dominating predictions (highest feature importance score of 0.0103).

Availability - The mobile application must remain functional for local features (inbox viewing, manual spam marking, blocking) even when internet connectivity is unavailable. API-dependent features must recover automatically when connectivity is restored.

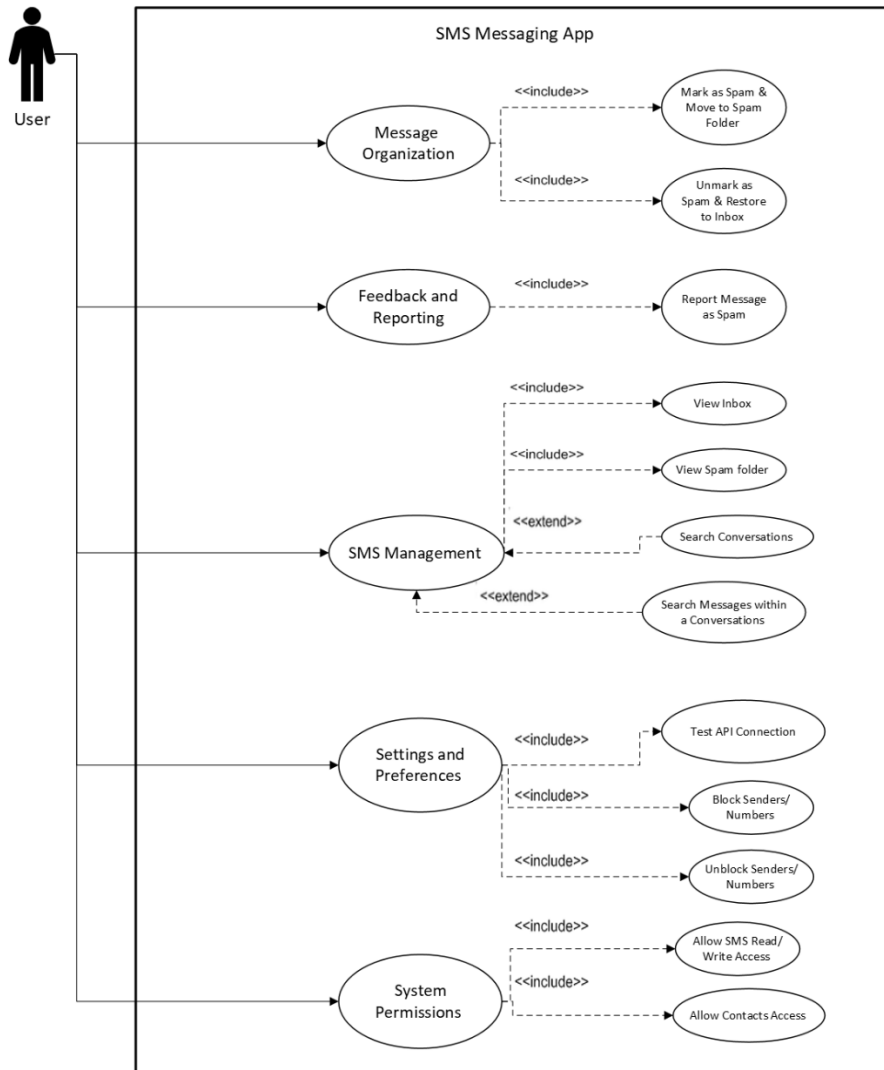
Maintainability: The codebase must be structured modularly to allow independent updates to the model, API, and mobile app. Retraining procedures must be executable without modifying the mobile application.

Scalability - The system architecture must support increased user load through containerized API deployment and cloud-based storage via Firebase Firestore.

Usability - Core features must be accessible within three user interactions. The interface must accommodate users with no technical background, using clear visual indicators, confirmation dialogs, and informative error messages throughout.

6. System Models

6.1 Use Case Diagram



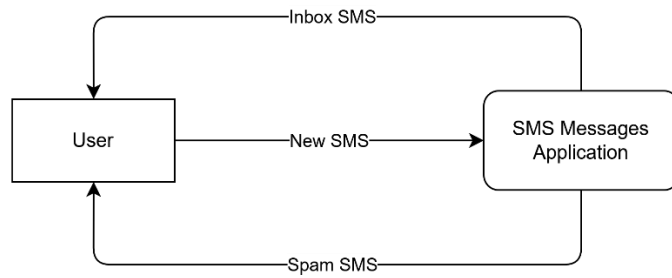
6.2 Use Case Descriptions

The system's use cases encompass the primary interactions between the user and the mobile application, including: automatic spam classification upon SMS receipt, manual marking/unmarking of spam, reporting spam to developers, blocking/unblocking senders, searching conversations and messages, viewing inbox and spam folder, and managing application settings and API connection.

6.3 Data Flow Diagrams (DFD)

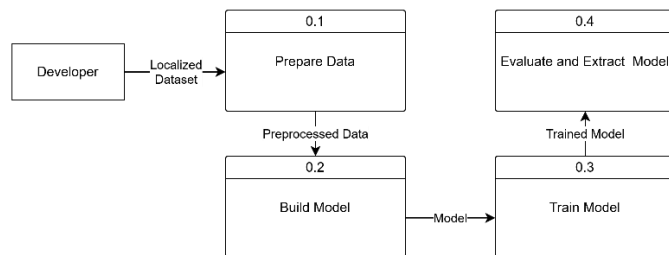
Data Flow Diagram (Level 0)

Level 0



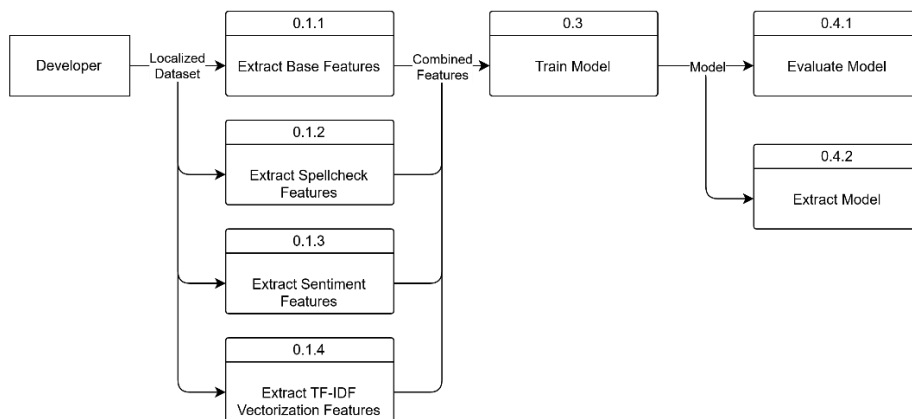
Data Flow Diagram (Level 1) [Random Forest Model]

Level 1



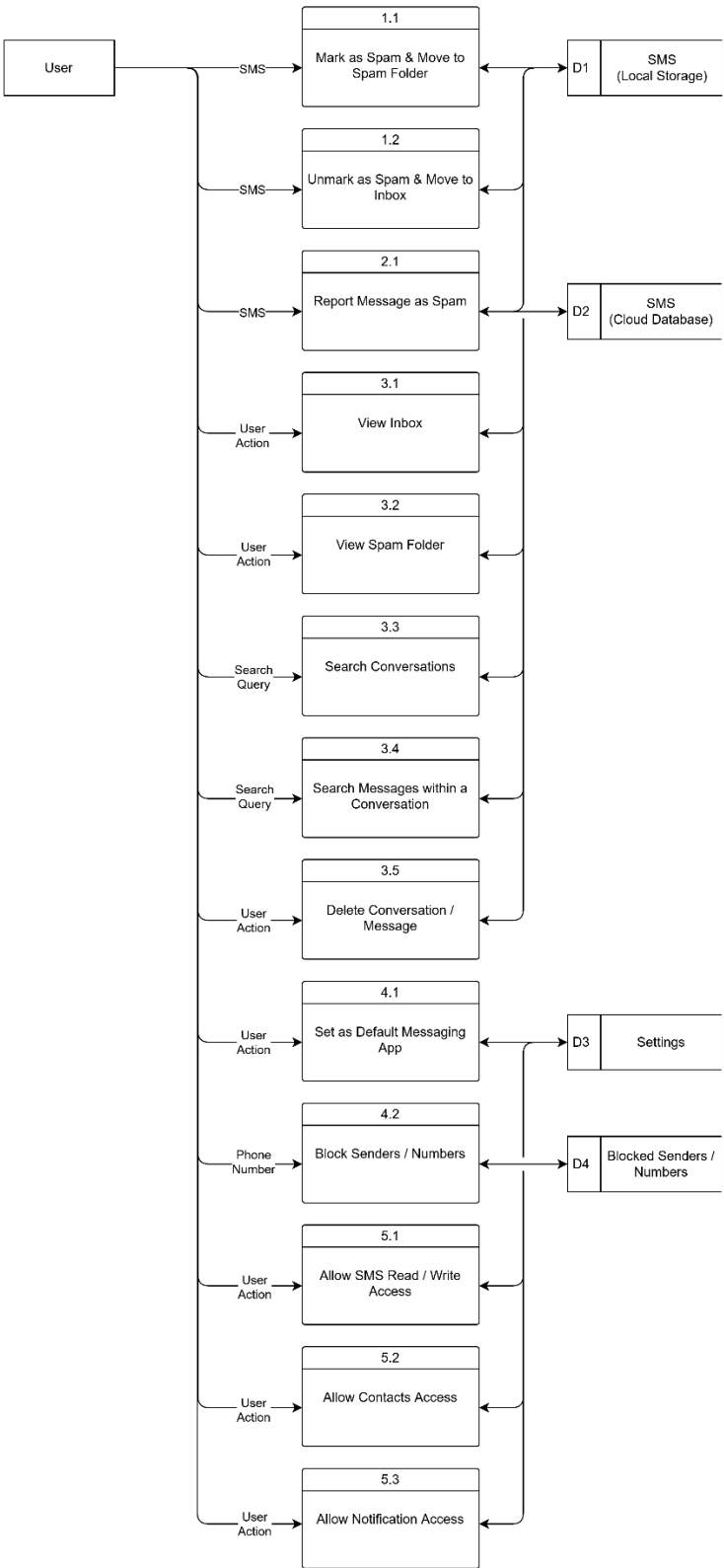
Data Flow Diagram (Level 2) [Random Forest Model]

Level 2



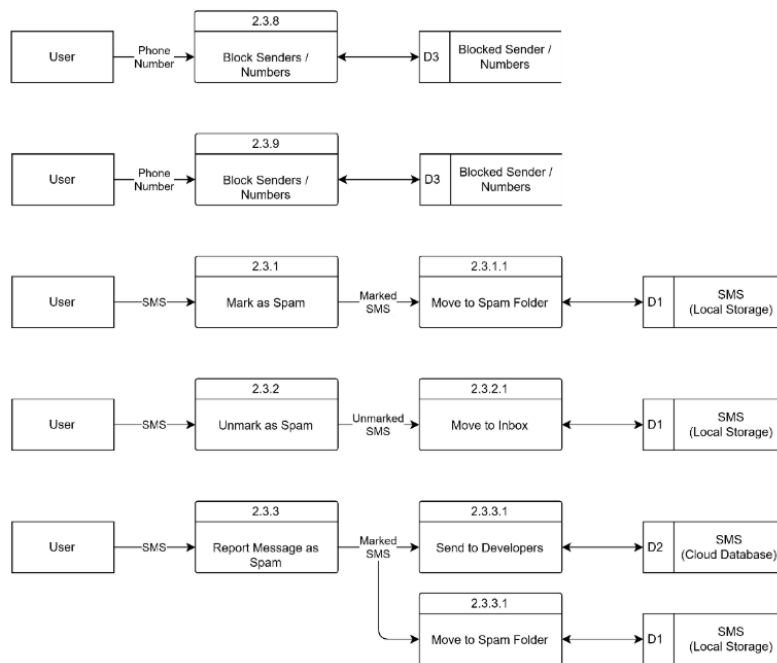
Data Flow Diagram (Level 1) [Mobile App]

Level 1



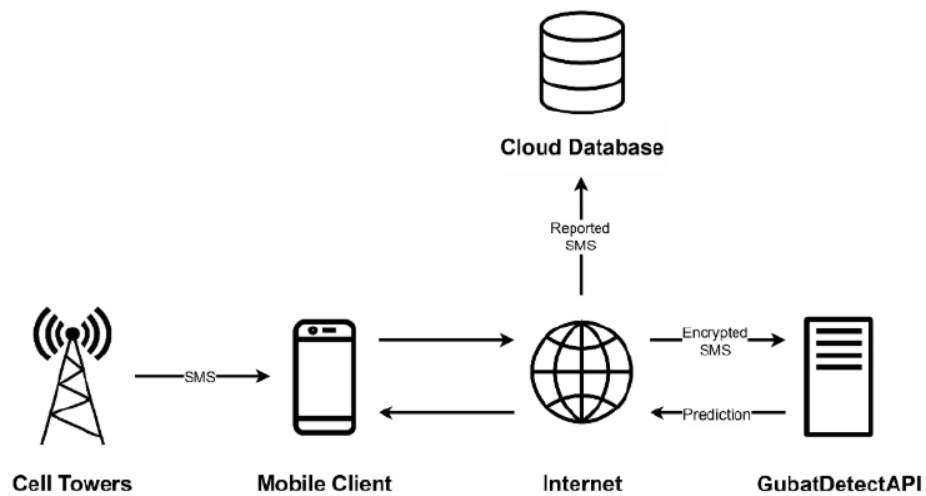
Data Flow Diagram (Level 2) [Mobile App]

Level 2

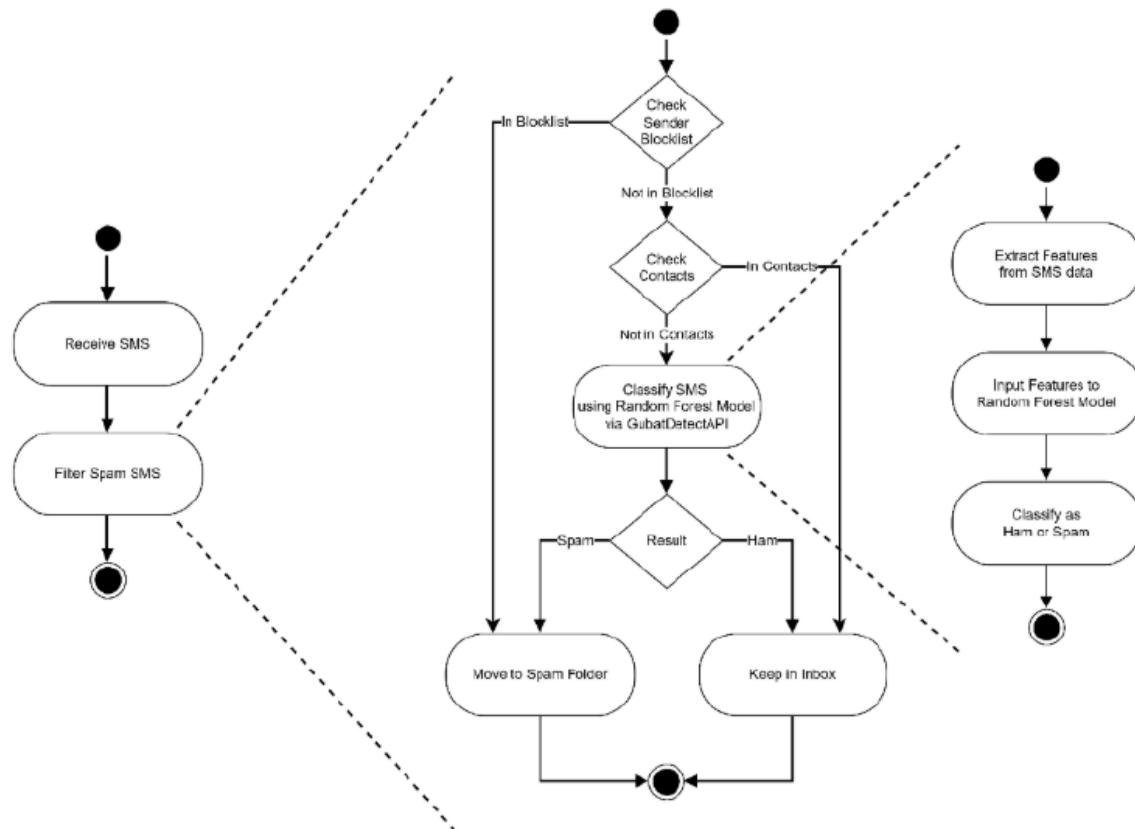


6.4 Sequence Diagrams

Client-Server Architecture Diagram



Activity Diagram



7. System Implementation Requirements

7.1 Development Environment

Component	Tool / Technology
Mobile App Framework	Flutter (Dart)
IDE	Visual Studio Code
Version Control	Git
Backend API Language	Python
API Framework	FastAPI
ML Library	Yggdrasil Decision Forests (YDF)
Feature Extraction	scikit-learn (StandardScaler, train_test_split) VADER (vaderSentiment)
Text Vectorization	TensorFlow Text Vectorization (TF-IDF)
Cloud Database	Google Firebase Firestore
Containerization	Docker

Secure Tunneling	ngrok
Target Platform	Android 13+

7.2 Coding Standards and Guidelines

All source code must follow consistent naming conventions across both the Flutter mobile app and the Python API. Flutter/Dart code must follow the official Dart style guide, using camelCase for variables and methods and PascalCase for class names. Python code must follow PEP 8 conventions, using snake_case for variables and functions.

All public functions, classes, and API endpoints must be documented with inline comments or docstrings. The machine learning pipeline, including preprocessing, feature extraction, model training, and evaluation steps, must be documented with inline comments explaining each step's purpose.

Sensitive values such as API keys, bearer tokens, and encryption keys must not be hardcoded in source files and must be managed via environment variables or secure configuration files excluded from version control.

7.3 Deployment Requirements

The GubatDetectAPI is deployed inside a Docker container, which packages the trained Random Forest model, preprocessing pipeline, and FastAPI application. The container is exposed externally through an ngrok secure tunnel, providing an HTTPS endpoint accessible by the mobile application.

Upon model retraining, the Docker container must be rebuilt and redeployed with the updated model artifacts. The mobile application's API endpoint URL must be updated if the ngrok tunnel URL changes between deployments.

The Flutter mobile application is distributed as an Android APK. Installation requires enabling installation from unknown sources on the target device or distribution via a managed enterprise channel.

7.4 Configuration Management

Source code for both the mobile application and GubatDetectAPI is managed using locally. The project repository contains separate directories for the Flutter app, the Python API, and the machine learning model training pipeline.

Model artifacts (trained YDF model, TF-IDF vocabulary, feature scaler, stop words) are stored alongside their corresponding dataset and hyperparameter configurations, allowing rollback to previous model versions if a retraining cycle produces degraded performance.

8. System Testing Requirements

8.1 Testing Strategy

The system employs the following testing levels across the development lifecycle:

Unit Testing was conducted incrementally across sprints, validating individual components including dataset integrity, model feature sets, data partitioning, fine-tuning results, and model inference on sample messages. See Appendix C for detailed unit test cases.

Integration Testing was performed, validating the integration of GubatDetectAPI with the mobile application, Google Firebase Firestore integration, and the end-to-end message classification flow from SMS receipt to folder assignment.

System Testing was after major development completion. The system was evaluated for functional correctness, API latency and reliability, and spam classification accuracy (number of misclassifications). System testing revealed that some spam messages containing malicious links bypassed the filter due to dataset recency limitations, documented as a known defect.

Acceptance Testing is evaluated against the performance criteria established in Section 5 and the acceptance criteria defined in Section 8.6.

8.2 Test Environment

Unit and integration tests for the machine learning pipeline were executed in a Python environment with the following components: YDF, scikit-learn, TensorFlow Text, vaderSentiment, and pandas. Model evaluation was performed on a held-out test set of 580 samples (20% of the 2,900-sample dataset).

Mobile application testing was conducted on physical Android devices running Android 13 or higher with active SIM cards. API connectivity testing was performed against the live GubatDetectAPI deployed in a Docker container via ngrok tunnel. Additional testing was done to check if the UI elements are all working such as buttons, screens and functions.

The Docker deployment environment was validated to confirm the container runs correctly with the final trained model and all required dependencies.

8.3 Test Cases and Scenarios

Test Case ID	Test Scenario	Test Steps	Test Data	Expected Results	Actual Results	Pass/Fail
PA1	App Launch - First Time Launch	1. Install app fresh on device 2. Launch the app	Fresh installation	Permission screen should display with progress indicator showing step 1 of 4	As Expected	Pass
PA2	App Launch - Returning User	1. Launch app with all permissions previously granted	User with granted permissions	App should bypass permission screen and navigate directly to HomeScreen	As Expected	Pass
PA3	Permission Request - SMS Permission	1. Tap "Grant Permission" on SMS step 2. Observe system permission dialog	SMS permission request	System dialog should appear requesting SMS access; upon granting, progress moves to step 2	As Expected	Pass
PA4	Permission Request - Contacts Permission	1. Tap "Grant Permission" on Contacts step 2. Observe system permission dialog	Contacts permission request	System dialog should appear requesting Contacts access; upon granting, progress moves to step 3	As Expected	Pass
PA5	Permission Request - Notification Permission	1. Tap "Grant Permission" on Notification step 2. Observe system permission dialog	Notification permission request	System dialog should appear requesting Notification access; upon granting, progress moves to step 4	As Expected	Pass
PA6	Permission Request - Internet hto Step	1. Tap "Continue" on Internet Connection info step 1. Tap "Deny" or any permission dialog	Info screen only Permission denied	App should navigate to HomeScreen App should remain on current permission step; user can retry	As Expected As Expected	Pass Pass
PA7	Permission Denied - Single Denial	2. Observe app behavior 1. Navigate to HomeScreen	No test data needed	Inbox tab should be selected by default; bottom navigation should show Inbox and Spam options	As Expected	Pass
PA8	Home Screen - Default View	2. Observe default tab 1. View inbox with no messages	No SMS messages on device	"No messages in inbox" text should display in center	As Expected	Pass
PA9	Home Screen - Empty Inbox	1. Navigate to Spam tab with no spam messages	No spam messages	"No spam messages" text should display in center	As Expected	Pass
PA10	Home Screen - Tab Navigation	1. Tap on Spam tab 2. Tap on Inbox tab	Existing messages	Active tab should highlight with green color and icon; inactive tab should show gray	As Expected	Pass
PA11	Home Screen - Pull to Refresh	1. Pull down on inbox list 2. Release to trigger refresh	Existing messages	Refresh indicator should appear; messages should reload from device	As Expected	Pass
PA12	Home Screen - Pull to Refresh	1. Pull down on inbox list 2. Release to trigger refresh	Existing messages	Refresh indicator should appear; messages should reload from device	As Expected	Pass
PA13	Conversation Tile Display	1. View conversation list in inbox	Messages from different senders	Each tile should show: contact name/phone number, latest message preview, timestamp, avatar with first letter	As Expected	Pass
PA14	Conversation Tile - Contact Name Display	1. View message from a saved contact	Message from contact in phone book	Contact display/name should appear instead of phone number	As Expected	Pass
PA15	Conversation Tile - Unknown Number Display	1. View message from unsaved number	Message from unknown sender	Full phone number should display as the title	As Expected	Pass
PA16	Conversation Tile - Spam Indicator	1. Navigate to Spam folder	Spam messages	Each tile should have a red "SPAM" badge visible	As Expected	Pass
PA17	Conversation Screen - Open Conversation	1. Tap on any conversation tile	Any existing conversation	ConversationScreen should open showing all messages from that sender in reverse chronological order	As Expected	Pass
PA18	Conversation Screen - Message Display	1. View individual messages in conversation	Multiple messages	Each message should display in a card with body text and formatted timestamp	As Expected	Pass
PA19	Conversation Screen - Read Only Indicator	1. View bottom of conversation screen	Any conversation	"Read Only" text with eye icon should display at bottom	As Expected	Pass
PA20	Conversation Screen - Search Within Conversation	1. Tap search icon in conversation 2. Enter search term	Search term matching message content	Only messages containing the search term should display; "No messages found" if no matches	As Expected	Pass
PA21	Conversation Screen - Search Clear	1. Perform search 2. Tap clear (X) button	Active search query	Search field should clear and all messages should reappear	As Expected	Pass
PA22	Conversation Screen - Back Navigation	1. Tap back arrow from conversation	Active conversation	Should return to previous screen (HomeScreen)	As Expected	Pass
PA23	Search Screen - Open Search	1. Tap search icon in app bar	From HomeScreen	SearchScreen should open with autofocus on search field	As Expected	Pass
PA24	Search Screen - Search by Content	1. Enter text that exists in message body	Search term: "promo"	Conversations containing messages with "promo" should appear	As Expected	Pass
PA25	Search Screen - Search by Phone Number	1. Enter partial or full phone number	Phone number: "0917"	Conversations from addresses containing "0917" should appear	As Expected	Pass
PA26	Search Screen - Search by Contact Name	1. Enter contact name	Contact name: "John"	Conversations from contacts with "John" in name should appear	As Expected	Pass
PA27	Search Screen - No Results	1. Enter non-matching search term	Search term: "xyz123nonexistent"	"No messages found" should display	As Expected	Pass
PA28	Search Screen - Empty State	1. Open search screen 2. Don't enter any text	No input	"Enter a search term" should display	As Expected	Pass
PA29	Selection Mode - Enter	1. Long press on any conversation tile	Any conversation	Selection mode should activate; checkbox/background highlight should appear; action buttons should appear in app bar	As Expected	Pass
PA30	Selection Mode - Toggle Selection	1. Long press to enter selection mode 2. Tap other conversations	Multiple conversations	Each tapped conversation should toggle selection state (selected/unselected)	As Expected	Pass
PA31	Selection Mode - Cancel	1. Enter selection mode 2. Tap back arrow	Active selection mode	Selection mode should exit; all selections should clear	As Expected	Pass
PA32	Selection Mode - Mark as Spam (inbox)	1. Select one or more conversations in inbox 2. Tap "Mark as Spam"	Selected non-spam messages	Confirmation dialog should appear with option to report spam	As Expected	Pass
PA33	Spam Confirmation Dialog - Mark as Spam	1. Trigger mark as spam action 2. Observe dialog content	Single conversation selected	Dialog should show: title "Mark as Spam", message about moving to spam folder, checkbox for reporting	As Expected	Pass
PA34	Spam Confirmation Dialog - Multiple Messages	1. Select multiple conversations 2. Trigger mark as spam	Multiple conversations selected	Dialog should use plural form: "Mark these messages as spam"	As Expected	Pass
PA35	Spam Confirmation Dialog - Cancel	1. Trigger mark as spam dialog 2. Tap "Cancel"	Active dialog	Dialog should close; no changes should occur	As Expected	Pass

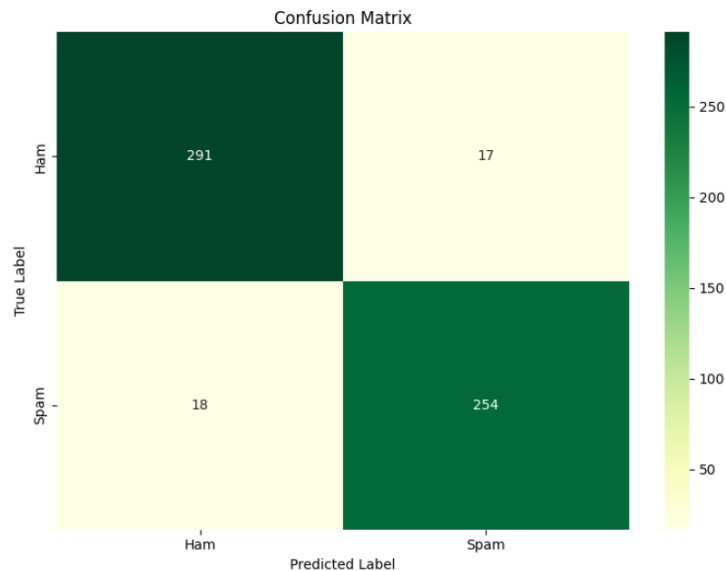
PA36	Spam Confirmation Dialog - Confirm Without Report	1. Trigger mark as spam dialog 2. Uncheck "Report this as spam" 3. Tap "Yes"	Active dialog	Messages should move to spam folder, success snackbar should display	As Expected	Pass
PA37	Spam Confirmation Dialog - Confirm With Report	1. Trigger mark as spam dialog 2. Check "Report this as spam" 3. Tap "Yes"	Active dialog with report checkbox	Messages should move to spam AND report to Firebase, success snackbar should display	As Expected	Pass
PA38	Umark as Spam (Spam Folder)	1. Navigate to Spam folder 2. Select conversation(s) 3. Tap "Umark as Spam"	Selected spam messages	Confirmation dialog should appear, upon confirmation, messages should restore to inbox	As Expected	Pass
PA39	Block Number from Selection	1. Enter selection mode in inbox 2. Select conversations 3. Tap "Block"	Selected conversations	Selected phone numbers should be added to blocked list, success snackbar should display with count	As Expected	Pass
PA40	Block Already Blocked Number	1. Select conversation from already blocked number 2. Tap "Block"	Already blocked number	"Selected number/s already blocked" message should display	As Expected	Pass
PA41	Blocked Numbers Screen - Open	1. Tap menu (three dots) 2. Select "Blocked Numbers"	From HomeScreen	BlockedNumbersScreen should open showing list of blocked numbers	As Expected	Pass
PA42	Blocked Numbers Screen - Empty State	1. Open with no blocked numbers	No blocked numbers	"No blocked numbers" text with block icon should display in center	As Expected	Pass
PA43	Blocked Numbers Screen - Add Number	1. Tap floating action button (+) 2. Enter valid Philippine number 3. Tap "Block"	Number: +639123456789	Number should be added to list, success snackbar "Number blocked successfully" should display	As Expected	Pass
PA44	Blocked Numbers Screen - Invalid Number Format	1. Tap floating action button (+) 2. Enter invalid number format 3. Tap "Block"	Number: 12345	Error message "Invalid number format" should display in dialog	As Expected	Pass
PA45	Blocked Numbers Screen - Empty Number	1. Tap floating action button (+) 2. Leave field empty 3. Tap "Block"	Empty input	Error message "Please enter a number" should display	As Expected	Pass
PA46	Blocked Numbers Screen - Duplicate Number	1. Try to add already blocked number	Already blocked number	"Number already blocked" snackbar should display	As Expected	Pass
PA47	Blocked Numbers Screen - Unblock	1. Tap delete icon on blocked number 2. Confirm unblock in dialog	Existing blocked number	Confirmation dialog should appear, upon confirming, number should be removed from list	As Expected	Pass
PA48	Blocked Numbers Screen - Unblock Cancel	1. Tap delete icon 2. Tap "Cancel" in dialog	Existing blocked number	Number should remain in blocked list	As Expected	Pass
PA49	Settings Screen - Open	1. Tap menu (three dots) 2. Select "Settings"	From HomeScreen	SettingsScreen should open with options	As Expected	Pass
PA50	Settings Screen - Manage Permissions	1. Tap "Manage Permissions" option	Settings screen	Device app settings should open	As Expected	Pass
PA51	Settings Screen - Test API Connection (Success)	1. Tap "Test API Connection" with working internet	API online	Snackbar should show "GabaDetectAPI is running (Xms)" with green background	As Expected	Pass
PA52	Settings Screen - Test API Connection (Failure)	1. Disable internet 2. Tap "Test API Connection"	No internet connection	Snackbar should show "GabaDetectAPI connection failed" with red background	As Expected	Pass
PA53	Incoming SMS - From Contact	1. Receive SMS from saved contact	SMS from contact	Message should appear in inbox, notification should show contact name	As Expected	Pass
PA54	Incoming SMS - Spam Detection	1. Receive SMS that API classifies as spam	Spam SMS content	Message should appear in Spam folder, spam notification should display	As Expected	Pass
PA55	Incoming SMS - From Blocked Number	1. Receive SMS from blocked number	SMS from blocked number	Message should automatically be marked as spam	As Expected	Pass
PA56	Incoming SMS - Offline Queue	1. Receive SMS while offline 2. Go back online	SMS while no internet	Message should initially go to inbox, when back online, should be reclassified if spam	As Expected	Pass
PA57	Offline Indicator	1. Launch app without internet connection	No internet	Red snackbar should display, "Cannot connect to GabaDetectAPI, spam messages might go to inbox"	As Expected	Pass
PA58	Notification - Ham Message	1. Receive legitimate message	Non-spam SMS	Notification should appear with green color showing sender name and message preview	As Expected	Pass
PA59	Notification - Spam Message	1. Receive spam message	Spam SMS	Notification should appear with red color and "Spam Detected" title	As Expected	Pass
PA60	Notification Tap	1. Tap on new message notification	Active notification	App should open to relevant conversation	As Expected	Pass
PA61	Contact Whitelist - Spam Detection Bypass	1. Receive message from saved contact	Message from contact	Message should go to inbox regardless of content (contacts are whitelisted)	As Expected	Pass
PA62	App Lifecycle - Resume	1. Put app in background 2. Send new SMS to device 3. Resume app	New SMS while backgrounded	Messages should refresh and show new messages	As Expected	Pass
PA63	Date Formatting - Today	1. View message received today	Message from today	Time should display as HH:MM format (e.g., "14:30")	As Expected	Pass
PA64	Date Formatting - Yesterday	1. View message from yesterday	Message from yesterday	"Yesterday" should display	As Expected	Pass
PA65	Date Formatting - This Week	1. View message from 3 days ago	Message from 3 days ago	"3d ago" should display	As Expected	Pass
PA66	Date Formatting - Older	1. View message from 2 weeks ago	Message from 2 weeks ago	Date should display as DD/MM/YYYY format	As Expected	Pass
PA67	Data Encryption - Local Storage	1. Check Hive database files	Encrypted Hive boxes	All local data (inbox, spam, blocked, numbers, quarant. messages) should be encrypted	As Expected	Pass
PA68	Data Encryption - API Transmission	1. Monitor network traffic	SMS message body	Message body should be AES-GCM encrypted before sending to API	As Expected	Pass
PA69	Firebase Spam Report	1. Trigger spam detection 2. Mark as spam with report option checked	Spam message	Entry should be created in Firestore, spam_reports collection with message and timestamp	As Expected	Pass
PA70	Firebase Spam Report - Retry	1. Report spam with intermittent connection	Unstable network	System should attempt up to 2 times with 500ms delay between attempts	As Expected	Pass

PA71	Queued Messages Processing	1. Have queued messages from offline 2. Restore internet connection	Queued messages pending	Pending messages should be processed and classified when connection restored	As Expected	Pass
PA72	Conversation Spam Propagation	1. Mark one message from sender as spam	Single message marked	All messages from same address should move to spam folder	As Expected	Pass
PA73	Conversation Unmark Propagation	1. Unmark sender from spam folder	Single sender unmarked	All messages from that address should restore to inbox	As Expected	Pass
PA74	Philippine Number Validation - Valid	1. Enter valid PH number in block dialog	+639123456789	Number should be accepted and added to blocked list	As Expected	Pass
PA75	Philippine Number Validation - Invalid Prefix	1. Enter number with wrong prefix	+63123456789	"Invalid number format" error should display	As Expected	Pass
PA76	Philippine Number Validation - Wrong Length	1. Enter number with wrong length	+6391234567	"Invalid number format" error should display	As Expected	Pass
PA77	Philippine Number Validation - Missing Plus	1. Enter number without +63	639123456789	"Invalid number format" error should display	As Expected	Pass
PA78	SnackBar Auto-Hide on Navigation	1. Trigger a snackbar 2. Navigate to different screen	Active snackbar	Snackbar should hide when navigating to new screen	As Expected	Pass
PA79	App Icon Display	1. Check app icon on device home screen 1. Observe app icons throughout	Installed app Any screen	Custom launcher icon should display (green message icon) Primary color should be green (#4CAF50), while backgrounds, proper Material 3 styling	As Expected	Pass
PA80	App Theme	1. Check app bar on HomeScreen	HomeScreen	"Gubai Messages" title should display	As Expected	Pass
PA81	App Bar Title	1. Launch app and observe initial load	App startup	Loading spinner with green color should display while initializing	As Expected	Pass
PA82	Loading State	1. Tap menu (three dots) on HomeScreen 2. Tap search icon 1. Open search screen 2. Type text 3. Clear text	HomeScreen menu Spam folder active Search query	"Settings" and "Blocked Numbers" options should appear SearchScreen should open with isSpamFolder=true; should search only spam messages Clear button (X) should only appear when search field has content	As Expected	Pass
PA83	Menu Options	1. Tap menu (three dots) on HomeScreen	HomeScreen menu		As Expected	Pass
PA84	Search from Spam Folder	1. Navigate to Spam tab 2. Tap search icon	Spam folder active		As Expected	Pass
PA85	Search Clear Button Visibility	1. Open search screen 2. Type text 3. Clear text	Search query		As Expected	Pass
PA86	Back Navigation from Search	1. Open search screen 2. Tap back arrow	Active search	Should return to HomeScreen	As Expected	Pass
PA87	Conversation from Search Results	1. Perform search 2. Tap on result	Search results	Should navigate to ConversationScreen for that conversation	As Expected	Pass
PA88	Multiple Selection - Block and Spam	1. Select multiple conversations 2. Block them first 3. Try to mark as spam	Already blocked numbers	Action should complete but message should reflect blocked status	As Expected	Pass
PA89	Background Message Handling	1. Receive SMS while app is in background/terminated	SMS while app not active	Background message handler should process and classify the SMS	As Expected	Pass
PA90	API Timeout Handling	1. Trigger spam detection with slow network	Slow network (>30 seconds)	Request should timeout after 30 seconds; message should queue for retry	As Expected	Pass
PA91	Health Check Endpoint	1. Observe API health check request	Test API Connection	GET request to /health endpoint with Bearer token should be made	As Expected	Pass
PA92	Spam Prediction API	1. Receive new SMS	New incoming SMS	POST request to /predict endpoint with encrypted message should be made	As Expected	Pass
PA93	Spam Detection Response Parsing	1. Receive API response with predictions	API response: {"predictions": [1, 0, 1]}	Predictions should parse: 1=spam, 0=not spam	As Expected	Pass
PA94	Contact Refresh	1. Add new contact on device 2. Pull to refresh in app	New contact added	ContactService should rebind contacts and display new contact names	As Expected	Pass
PA95	Notification Cancel	1. Trigger notification 2. Open app	Active notification	Notification should be dismissable	As Expected	Pass
PA96	Environment Variables	1. Check app configuration	env file	API_URL, ENCRYPTON_KEY, API_KEY, HIVE_KEY should be loaded from env file	As Expected	Pass
PA97	Hive Box Persistence	1. Add messages to inbox/spam 2. Close and reopen app	Messages in storage	All data should persist and be available after app restart	As Expected	Pass
PA98	Verify API Health Endpoint (API-Sided)	1. Start the server/container. 2. Send GET request to /health. 3. Include valid APIKey in Header.	Authorization: Bearer <API_KEY>	Status Code: 200 OK Body: {"status": "healthy", "message": "GubaiFilterAPI is running"}	As Expected	Pass
PA99	Verify Authentication Failure (API-Sided)	1. Send GET request to /health. 2. Include invalid APIKey in Header.	Authorization: Bearer invalid_key	Status Code: 401 Unauthorized	As Expected	Pass
PA100	Predict Spam Message (Encrypted) (API-Sided)	1. Encrypt text "PROMO: Win cash now!" using AES-256-GCM.	{"messages": [{"encrypted_string"}]}	Status Code: 200 OK Prediction: 1 (Spam)	As Expected	Pass
PA101	Predict Ham Message (Encrypted) (API-Sided)	1. Encrypt text "Hi Goodmorning!" using AES-256-GCM. 2. Send POST to /predict with encrypted payload.	{"messages": [{"encrypted_string"}]}	Status Code: 200 OK Prediction: 0 (Ham)	As Expected	Pass
PA102	Verify Docker Container Startup	1. Build Docker image. 2. Run container on port 8000. 3. Check logs	docker build -t gubai-server. docker run -p 8000:8000 gubai-server	Container starts successfully. Logs show: Unicom running on http://0.0.0.0:8000	As Expected	Pass
PA103	Verify Batch Prediction (API-Sided)	1. Encrypt two messages (1 Spam, 1 Ham). 2. Send POST to /predict with list of both.	{"messages": [{"encrypted_spam"}, {"encrypted_ham"}]}	Status Code: 200 OK Returns list of 2 predictions	As Expected	Pass
PA104	Verify Missing Environment Variables	1. Run server without API_KEY or ENCRYPTON_KEY set.	python server.py	Server should raise Error or fail to start due to missing env vars.	As Expected	Pass

8.3.1 Model Results

This section provides the raw performance results of the trained Random Forest model. The evaluation metrics (Confusion Matrix, Classification Report, and Feature Importance using Mean Decrease in Accuracy) was applied to provide a complete assessment of the model's spam detection capabilities.

Confusion Matrix



Key results:

1. A total of 546 including 291 ham and 254 spam messages have been correctly classified, representing 93.97% of all test samples.
2. 17 ham messages were incorrectly classified as spam, representing a false positive rate of 5.52%.
3. 18 spam messages were incorrectly classified as ham, representing a false negative rate of 6.62%.

Classification Report

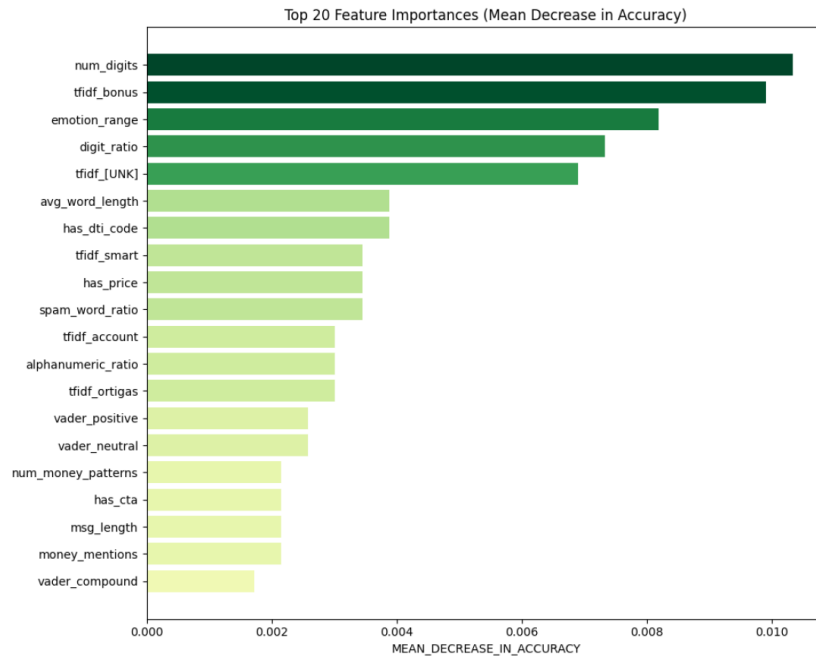
	Precision	Recall	F1-Score	Support
Ham (0)	0.9417	0.9448	0.9433	308
Spam (1)	0.9373	0.9338	0.9355	272
Accuracy			0.9397	580
Macro Avg	0.9395	0.9393	0.9394	580
Weighted Avg	0.9396	0.9397	0.9396	580

Key results:

1. 546 correct classifications out of 580 total test samples, an overall accuracy of 93.97%. Exceeding the target accuracy of 90% by 3.97%.

2. A less than 1% F1-Score difference between classes with 0.78% (94.33% - 93.55%), indicating a balanced performance despite the slight class imbalance.

Feature Importance using Mean Decrease in Accuracy



Key results:

1. The presence of digits is the most effective feature in determining spam SMS.
2. Keywords such as “bonus” and “smart” are some of the frequent words that spam senders used.
3. “tfidf_[UNK]” is the unknown words encountered that was not included in TF-IDF vocabulary when fitted. This feature is the 5th most important feature that contributes to model’s accuracy.
4. The model can detect “emotion_range” or emotional balance of the text through sentiment analysis, which is the 3rd most effective feature.
5. The model does not rely on a single dominant feature with the highest only being 0.0103.

8.4 Requirements Traceability Matrix (RTM)

Requirement ID	System Feature	Linked Test Cases	Status
REQ-01	Mark as Spam & Move to Spam Folder	PA29, PA30, PA31, PA32, PA33, PA34, PA35, PA36, PA37, PA69, PA72, PA78, PA88	Pass
REQ-02	Unmark as Spam & Restore to Inbox	PA29, PA30, PA31, PA38, PA73	Pass
REQ-03	Report Message as Spam	PA33, PA37, PA56, PA57, PA68, PA69, PA70, PA71	Pass

Requirement ID	System Feature	Linked Test Cases	Status
REQ-04	View Inbox	PA8, PA9, PA11, PA12, PA13, PA14, PA15, PA17, PA18, PA19, PA62, PA63, PA64, PA65, PA66, PA97	Pass
REQ-05	View Spam Folder	PA10, PA11, PA16, PA17, PA18, PA97	Pass
REQ-06	Search Conversations	PA23, PA24, PA25, PA26, PA27, PA28, PA84, PA86, PA87	Pass
REQ-07	Search Messages within a Conversation	PA20, PA21, PA85	Pass
REQ-08	Block Senders/Numbers	PA39, PA40, PA41, PA42, PA43, PA44, PA45, PA46, PA55, PA67, PA74, PA75, PA76, PA77, PA83, PA88	Pass
REQ-09	Unblock Senders/Numbers	PA41, PA42, PA47, PA48, PA83	Pass
REQ-10	Test API Connection	PA49, PA51, PA52, PA90, PA91, PA92, PA96, PA98, PA99, PA100, PA101, PA102, PA103, PA104	Pass
REQ-11	Allow SMS Access	PA1, PA2, PA3, PA7, PA50, PA53, PA54, PA89, PA93	Pass
REQ-12	Allow Contacts Access	PA1, PA2, PA4, PA7, PA14, PA50, PA61, PA94	Pass
REQ-13	Allow Notifications Access	PA1, PA2, PA5, PA7, PA50, PA53, PA58, PA59, PA60, PA95	Pass

8.5 Defect Management

Defects identified during testing are classified by the following severity levels:

- Critical - Defects that cause system crashes, data loss, or complete feature failure (e.g., API connection failure with no fallback, SMS permissions crash).
- High - Defects that cause incorrect core behavior, such as spam messages consistently misclassified or messages not moving to the correct folder.
- Medium - Defects that degrade user experience but do not prevent feature use, such as UI layout issues or delayed notifications.
- Low - Minor defects such as typographical errors, cosmetic issues, or non-blocking edge case behaviors.

Bug reports must include, a description of the defect, steps to reproduce, expected vs. actual behavior, severity level, and the sprint/build in which it was discovered. Defects

are tracked and resolved before the affected feature is considered complete for that sprint cycle.

8.6 Acceptance Criteria

Functional Acceptance - All 13 system features defined in Section 3 must perform as specified in their respective Stimulus/Response Sequences and Functional Requirements without critical defects.

Performance Acceptance -

- The trained Random Forest model must achieve a minimum accuracy of 90% on the test dataset.
- The weighted F1-Score must fall within 85–95%.
- Incoming SMS messages must be classified within 2 seconds per message.

Security Acceptance -

- All API communication must use HTTPS with HTTP Bearer key authentication.
- SMS data transmitted to GubatDetectAPI must be AES-256-GCM encrypted.
- Spam reports must be anonymous, containing only the message body.
- The blocked numbers list must be stored in encrypted local storage.

References

- Abid, M. A., Ullah, S., Siddique, M. A., Mushtaq, M. F., Aljedaani, W., & Rustam, F. (2022). Spam SMS filtering based on text features and supervised machine learning techniques. *Multimedia Tools and Applications*. <https://doi.org/10.1007/s11042-022-12991-0>
- Balita, C. (2024, April 29). Philippines: most frequent consumer fraud schemes 2021. Statista. <https://www.statista.com/statistics/1271755/philippines-most-frequent-consumer-fraud-schemes/>
- Dela Cruz, R. C. (2024). Globe, Smart block scam numbers amid unabated spam messages. Pna.gov.ph. <https://www.pna.gov.ph/articles/1229353>
- Hosseinpour, S., & Shakibian, H. (2024). Complex-network based model for SMS spam filtering. *Computer Networks*, 255, 110892. <https://doi.org/10.1016/j.comnet.2024.110892>
- Howe, S. (2025, June 18). Social Media Statistics in the Philippines. Meltwater. <https://www.meltwater.com/en/blog/social-media-statistics-philippines>
- InsiderPH. (2024). Philippines hit by 6.1M SMS scams, 610K scam calls in 2024 — Whoscall report. Insiderph.com. <https://insiderph.com/philippines-hit-by-61m-sms-scams-610k-scam-calls-in-2024-whoscall-report>
- Kemp, S. (2025, November 5). DataReportal – Global Digital Insights. DataReportal – Global Digital Insights. <https://datareportal.com/reports/digital-2026-philippines>
- Liakh, I., Chorniy, A., Lutak, O., & Tsenkner, M. (2025). Adaptive hybrid SMS spam detection system with user feedback-based self-learning. *Bulletin of Cherkasy State Technological University*, Vol. 30 No. 3. 2025, 121–132. <https://doi.org/10.62660/bcstu/3.2025.121>
- Lota, L. N., & Hossain, B. M. M. (2017). A Systematic Literature Review on SMS Spam Detection Techniques. *International Journal of Information Technology and Computer Science*, 9(7), 42–50. <https://doi.org/10.5815/ijitcs.2017.07.05>
- Mathieu Guillaume-Bert, Bruch, S., Stotz, R., & Pfeifer, J. (2023). Yggdrasil Decision Forests: A Fast and Extensible Decision Forests Library. ArXiv (Cornell University), KDD '23, August 6–10, 2023. <https://doi.org/10.1145/3580305.3599933>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12(85), 2825–2830.
- ScamWatchHQ. (2025, October 19). Philippines Scams 2025: Second-Highest Global Fraud Rate Sparks National Crisis Response. ScamWatchHQ. <https://www.scamwatchhq.com/philippines-scams-2025-second-highest-global-fraud-rate-sparks-national-crisis-response/>
- scikit-learn. (2019). StandardScaler. Scikit-Learn.org. <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html>
- stopwords-iso. (2025). stopwords-iso/CREDITS.md at master · stopwords-iso/stopwords-iso. GitHub. <https://github.com/stopwords-iso/stopwords-iso/blob/master/CREDITS.md>

Appendices

Appendix A: Glossary

Definitions of technical terms and acronyms

Term	Definition
Ham	A legitimate, non-spam SMS message
Spam	An unsolicited, potentially malicious or fraudulent SMS message
Smishing	SMS-based phishing, a form of social engineering using text messages to deceive users
Random Forest	An ensemble machine learning algorithm that builds multiple decision trees and aggregates their predictions
YDF (Yggdrasil Decision Forests)	A high-performance C++ library for decision forest algorithms used to implement the Random Forest model
TF-IDF	Term Frequency-Inverse Document Frequency, a text vectorization method that reflects how important a word is to a document within a corpus
VADER	Valence Aware Dictionary and sEntiment Reasoner, a lexicon-based sentiment analysis tool used for feature extraction
GubatDetectAPI	The project's custom REST API that receives encrypted SMS messages and returns a spam/ham classification prediction
Flutter	Google's open-source UI software development kit used to build the Android mobile application
FastAPI	A modern Python web framework used to develop GubatDetectAPI
ngrok	A secure tunneling service that exposes the locally hosted GubatDetectAPI to the internet via HTTPS
Docker	A containerization platform used to package and deploy GubatDetectAPI and the trained model
Firebase Firestore	Google's cloud-hosted NoSQL database used to store anonymous spam reports submitted by users
AES-256-GCM	Advanced Encryption Standard with 256-bit key in Galois/Counter Mode, the encryption method used for securing SMS data in transit
OOB (Out-of-Bag)	A validation method in Random Forest where samples not used in a tree's bootstrap sample are used to evaluate that tree's performance
Obfuscation	The practice of disguising spam keywords using character substitution (e.g., "b0nus" for "bonus") to evade text-based filters
Taglish	A code-switched language mixing Filipino (Tagalog) and English, commonly used in Philippine SMS communication
F1-Score	The harmonic mean of precision and recall; used as the primary optimization metric during model fine-tuning
Precision	The proportion of predicted spam messages that are actually spam (minimizes false positives)
Recall	The proportion of actual spam messages that are correctly identified (minimizes false negatives)

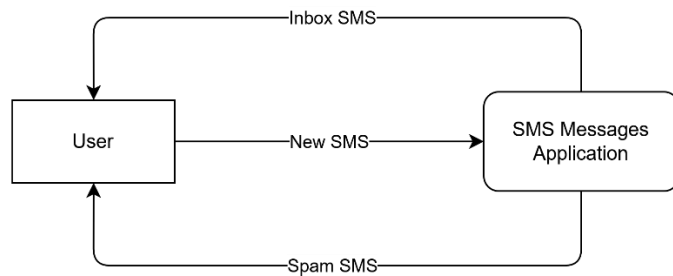
Scrum	An Agile framework for managing and completing complex projects through iterative sprint cycles
-------	---

Appendix B: Analysis Models

Additional diagrams or tables

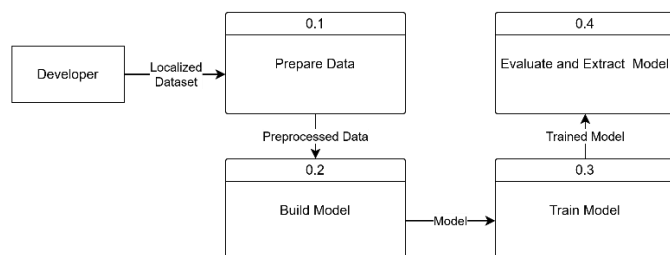
Data Flow Diagram (Level 0)

Level 0



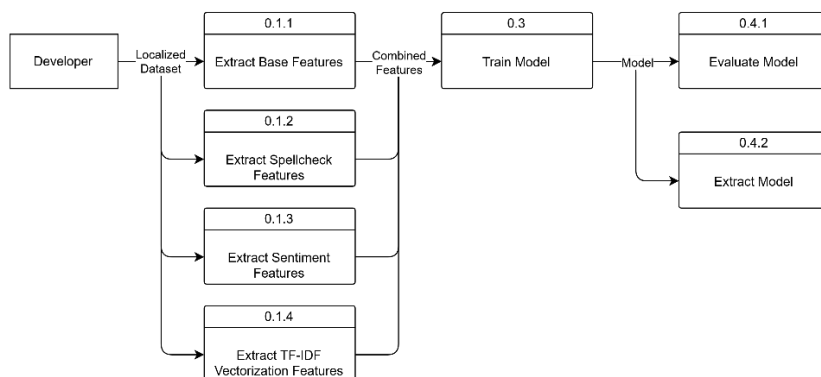
Data Flow Diagram (Level 1) [Random Forest Model]

Level 1

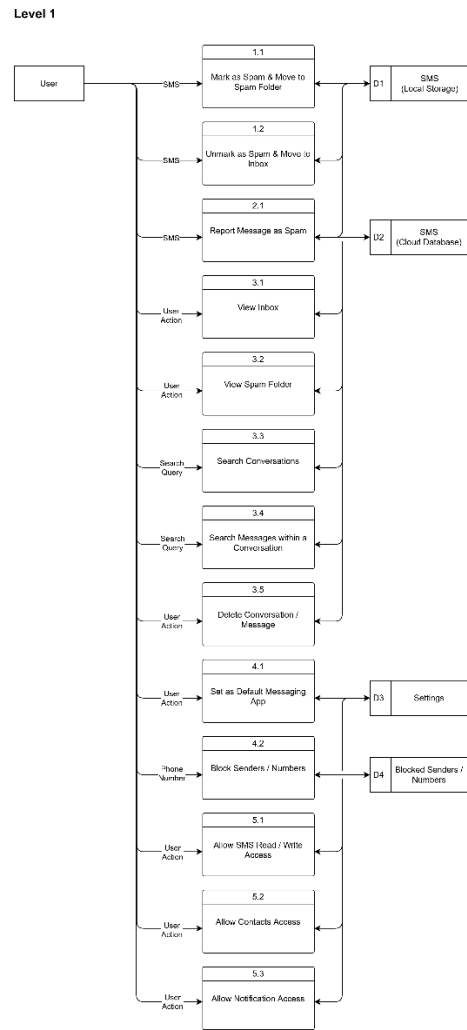


Data Flow Diagram (Level 2) [Random Forest Model]

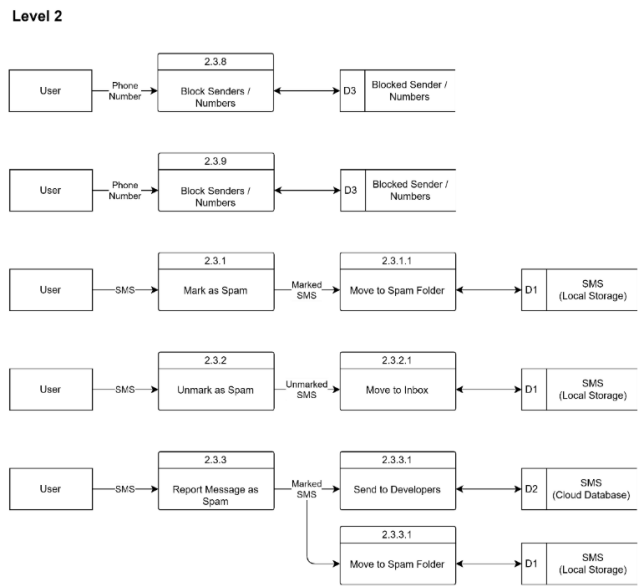
Level 2



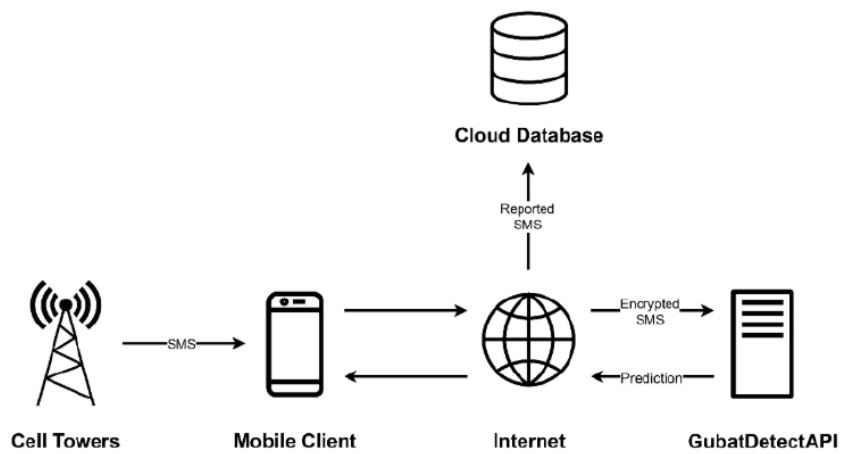
Data Flow Diagram (Level 1) [Mobile App]



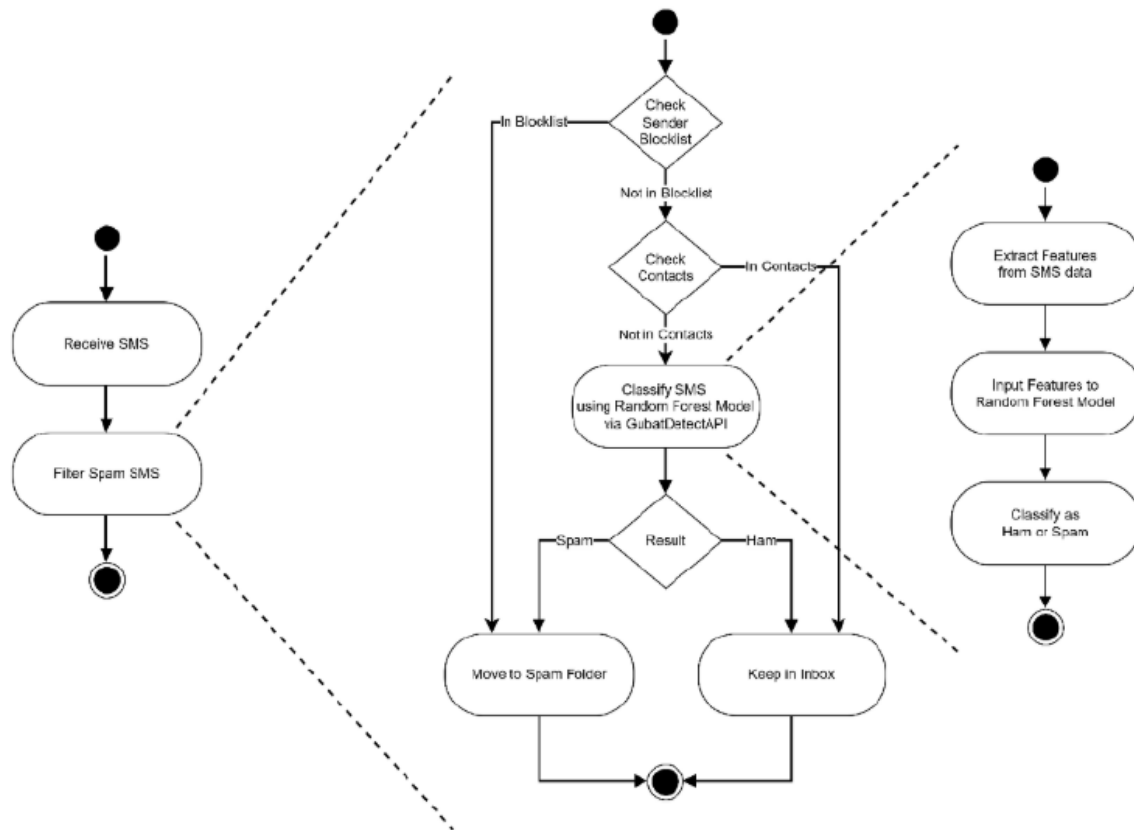
Data Flow Diagram (Level 2) [Mobile App]



Client-Server Architecture Diagram



Activity Diagram





SMS Access

Required to read existing and incoming messages

Grant Permission



Contacts Access

Required to identify and whitelist all contacts known contacts

Grant Permission



Notification Permission

Required to enable notifications when new messages are received

Grant Permission

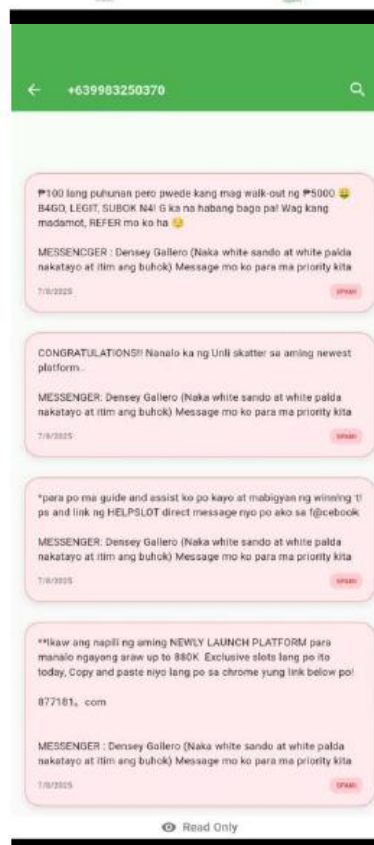
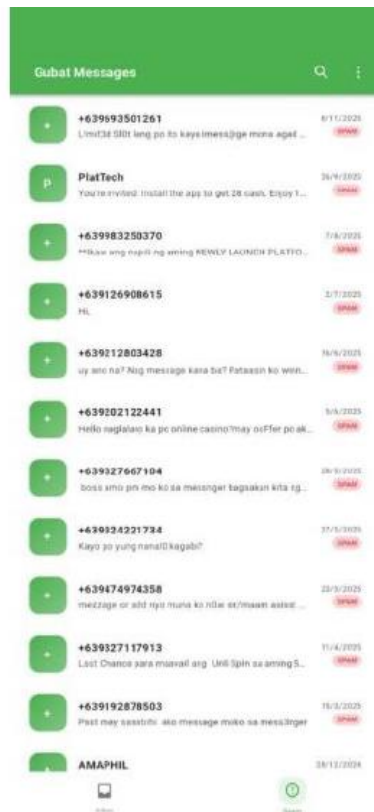
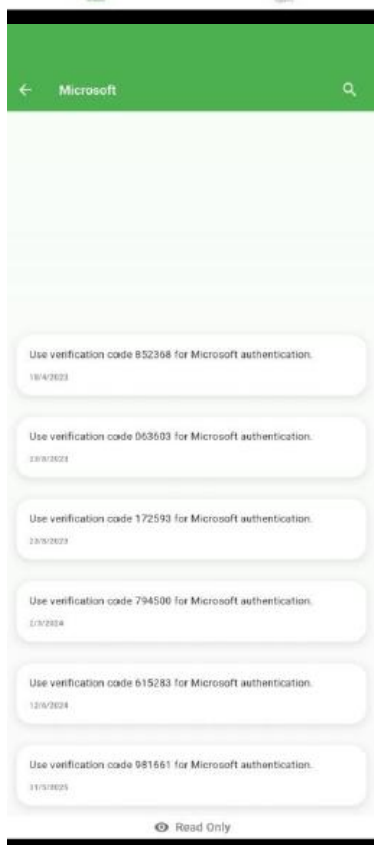


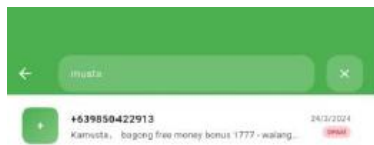
Internet Connection Required

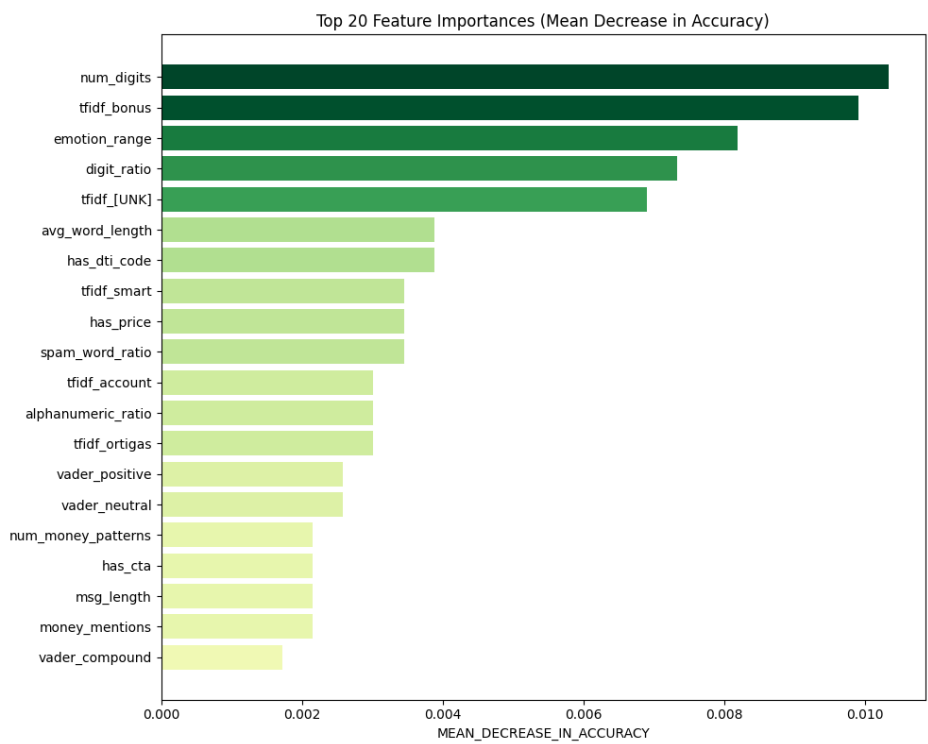
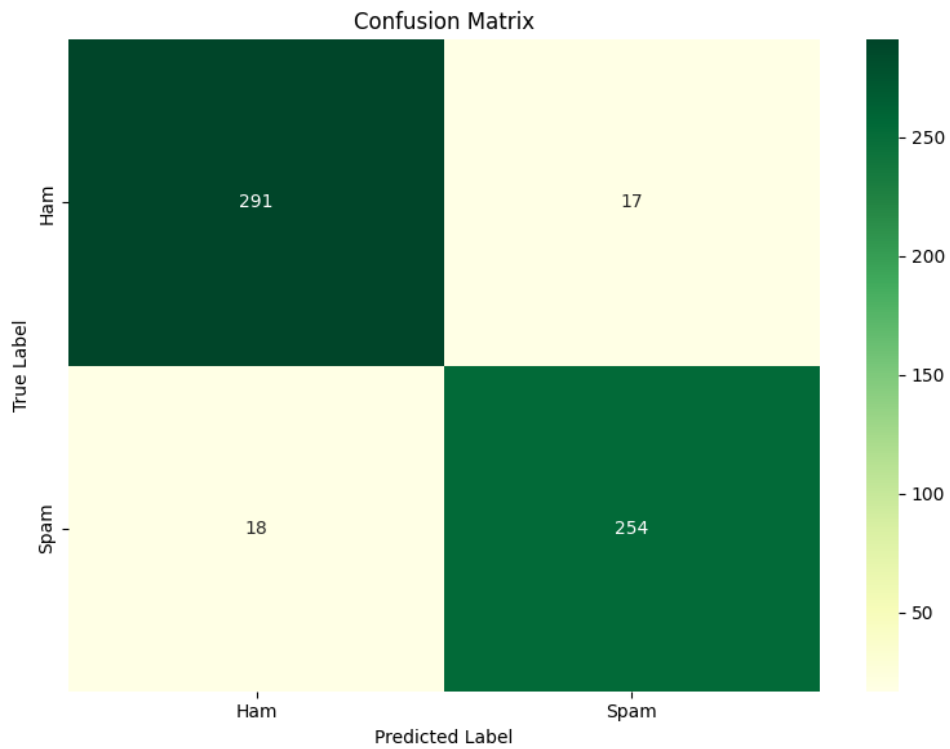
Our spam filter requires internet connection. Other core functionalities still work offline
*All SMS data are encrypted before sending to GubatDetectAPI

Continue









Appendix C: Sample Test Cases

Dataset Shape and Class Distribution

```
▼ ... Loading data from: /content/drive/MyDrive/colab_mounted_files/cleaned-final-212-shuffled
Data loaded successfully!
Dataset shape: (2900, 3)
Columns: ['sender', 'text', 'label']

Class distribution:
  Spam (1): 1360 messages
  Ham (0): 1540 messages
```

Feature set and Data Partition

```
Extracting features from messages...
Preparing features...
Extracting VADER sentiment features...
Extracting TF-IDF features...
Total features extracted: 3037
- Base features: 31
- Sentiment features: 6
- TF-IDF features: 3000

Splitting data...
Training set: 2320 samples
Test set: 580 samples
```

Fine-Tuning Top Result

```
{
  "config": {
    "tfidf_features": 3000,
    "ngrams": "1",
    "num_trees": 500,
    "max_depth": -1
  },
  "accuracy": 0.9313793103448276,
  "precision": 0.9313793103448276,
  "recall": 0.9313793103448276,
  "f1_score": 0.9313793103448276,
  "training_time": 994.5882747173309,
  "status": "success"
},
```

Sample Messages inputted into the Trained Model

```
=== Prediction Results ===

...

Message 1:
Text: "Pamessage nako dahil Napili !tong number mo na makakuha ng E@SY SCATT3R at 7,77
Prediction: SPAM
Spam Probability: 61.60%
Confidence: 61.60%

Message 2:
Text: "You received PHP7,045.95 in your Wallet from 747 Live. Your balance is PHP7045.
Prediction: HAM
Spam Probability: 3.30%
Confidence: 96.70%

Message 3:
Text: "SERVICE ADVISORY

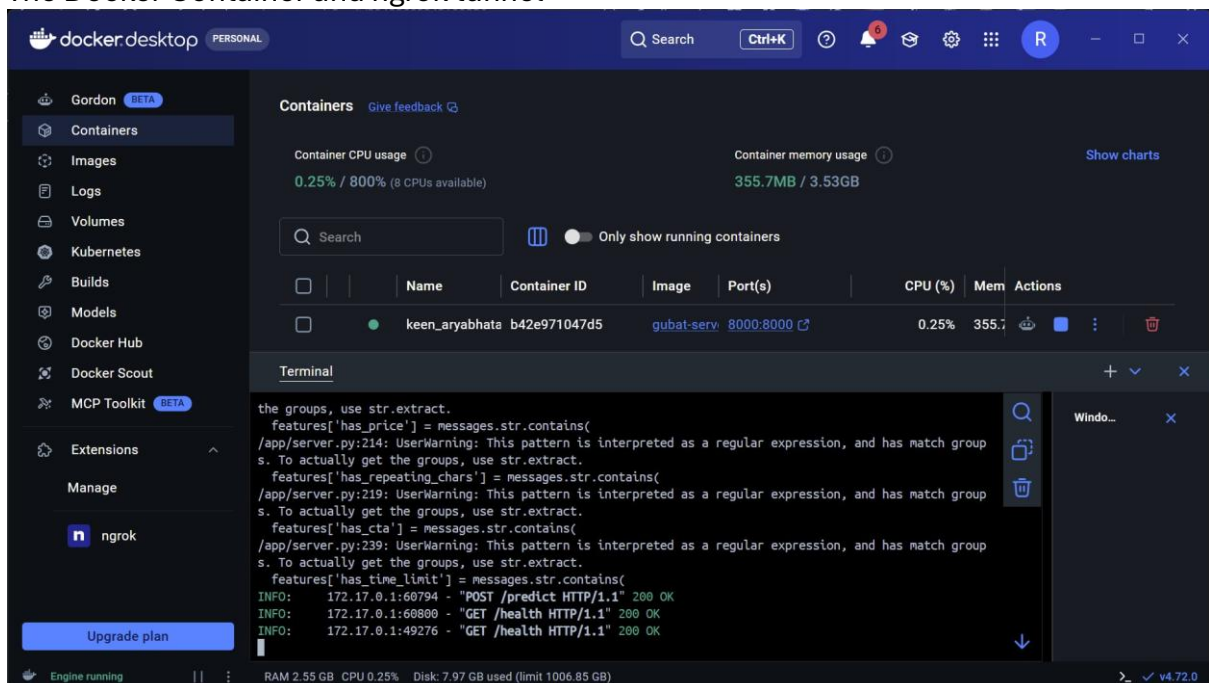
There will be a scheduled maintenance activity in your area from OCTOBER 27 1AM to..."
Prediction: HAM
Spam Probability: 8.40%
Confidence: 91.60%

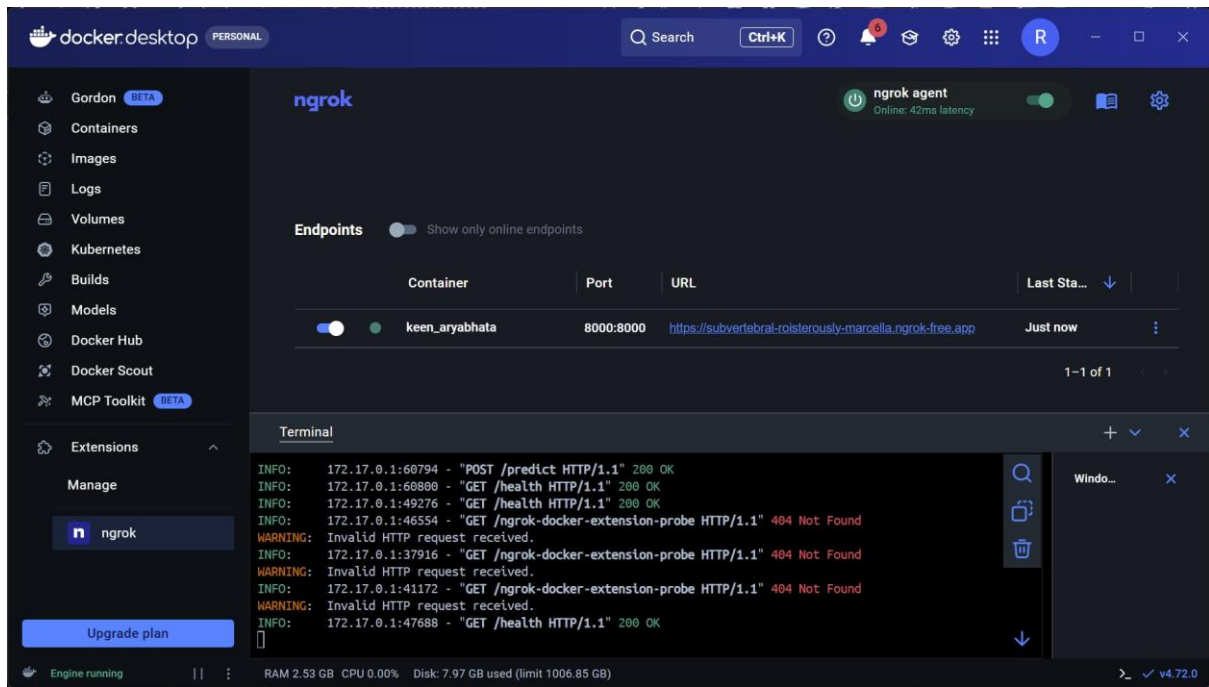
Message 4:
Text: "FREE HEALTH INSURANCE on GCash with SMART!

Stay connected & protected when you subscribe to POWER A..."
Prediction: HAM
Spam Probability: 1.60%
Confidence: 98.40%

Message 5:
Text: "Save more with Smart and Maya this payday weekend!"
```

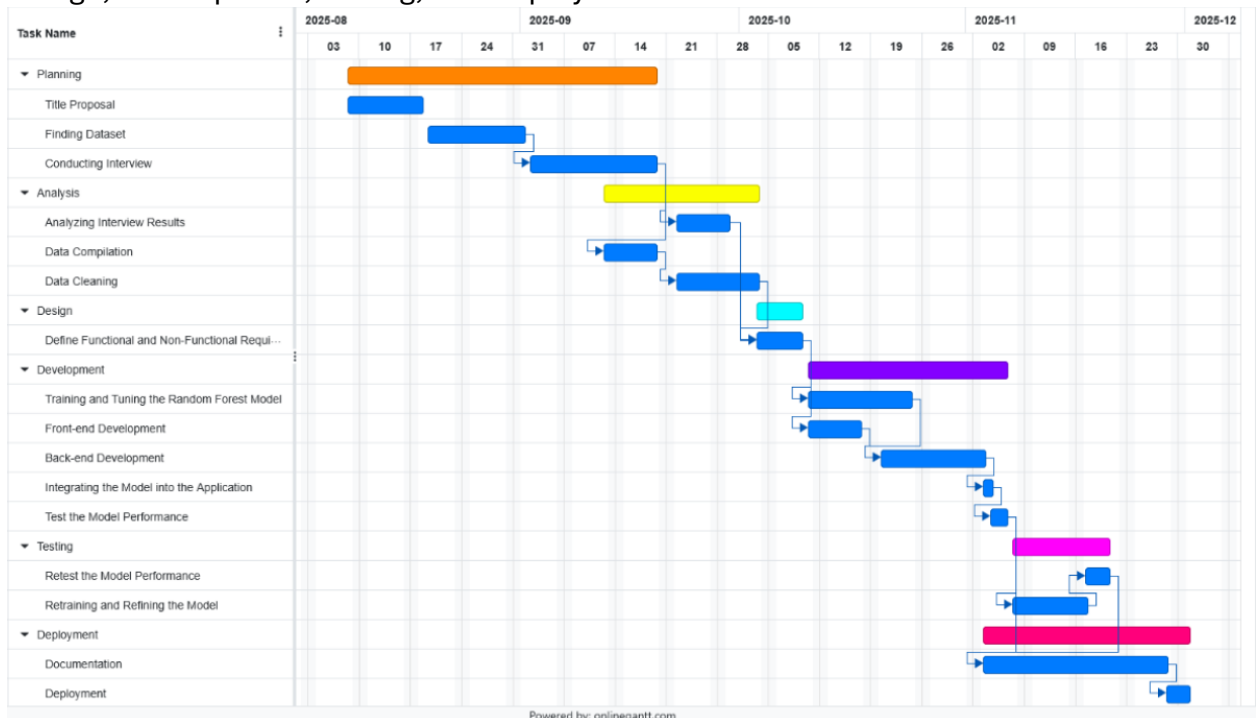
The Docker Container and ngrok tunnel





Appendix D. SYSTEM IMPLEMENTATION PLAN

The project was developed over approximately 4 months following the Scrum Agile methodology, divided into 7 sprints. Development phases included Planning, Analysis, Design, Development, Testing, and Deployment.



And additional of approximately 4 months for further testing.

[illegible]

Appendix E. SYSTEM MAINTENANCE & SUPPORT

The following procedure is used to retrain the model to adapt to evolving spam tactics, in this order:

1. Export collected spam reports from Firestore Database.
2. Label and clean according to Dataset Collection and Cleaning.
3. Merging the new data with the localized dataset.
4. Retrain model with the new dataset, adjusting the data preprocessing, feature extraction, hyperparameters as needed.
5. Evaluate and compare the performance of the retrained model against previous model according to Evaluation Metrics.
6. Rebuild and redeploy via Docker container with the updated model if the new model passes the maintainability requirements.

Maintenance Schedule - Model retraining must be performed on a quarterly basis (every 90 days) with a minimum of 10% new samples relative to the current dataset size. The retrained model must not be deployed if its accuracy falls more than 2% below the original model's benchmark performance.

Support Responsibilities - Developer-level support covers model retraining, API redeployment, Firebase database management, and bug resolution. End users are not expected to perform any maintenance actions. If the ngrok tunnel URL changes between deployments, the mobile application's configured API endpoint must be updated and redistributed as an updated APK.

Known Limitation for Maintenance - System testing revealed that spam messages containing malicious links may bypass the current filter due to the training dataset lacking sufficiently recent samples (data primarily from 2023–2024). This is a targeted area for improvement in the next retraining cycle, prioritizing collection of newer spam samples.

Appendix F Future Enhancements

The following enhancements are recommended for future development iterations:

1. Expanded Dataset Coverage - The current dataset contains samples primarily from 2023–2024. Future versions should incorporate more recent spam samples to address evolving tactics, particularly those involving malicious links that bypassed the current filter during system testing.
2. On-Device Model Deployment - Due to limited TensorFlow Lite support for YDF on Android, classification currently requires an internet connection. Future work should explore alternative lightweight model formats (e.g., ONNX or custom inference engines) to enable fully offline spam detection.
3. Multi-Language Support - The current model is optimized for Filipino-English (Taglish). Support for regional Philippine languages (e.g., Cebuano, Ilocano) could be introduced by expanding the dataset and stop word lists.

4. iOS Support - The application is currently Android-only. A cross-platform Flutter implementation targeting iOS would expand accessibility to a wider user base.
5. Adaptive Threshold Tuning - Based on the SE1 interview with Mr. Romnick Belen, future versions should support adjustable classification thresholds (e.g., shifting from 0.5 to 0.7) to allow users or administrators to balance false positive and false negative rates based on their risk preference.
6. Enhanced Reporting Analytics - Aggregated and anonymized spam report data stored in Firebase Firestore could be used to generate trend reports identifying emerging spam patterns for proactive model updates.

Appendix G Expert Interviews

Date of Interview: October 19, 2025

Interviewee Name: Dr. Elmer Matel, Director, Lifelong Learning, and Digital Education Lyceum of the Philippines University-Cavite

Interviewer Name: Aeron Valzado; Yoshida Al Zarouni, Dominc Torres

Question	Answer
Do you think it's beneficial for Filipino citizens to have a SMS filter trained on a local dataset?	It would be very beneficial for Filipino citizens to have an SMS filter trained on a local dataset. In the Philippines, many people receive scam or spam texts such as fake GCash alerts, raffle winnings from Shopee or Lazada, or parcel delivery scams that require advance payment. Since these messages are often written in Taglish or contain local slang, global spam filters trained mostly on English may not detect them. A locally trained filter would better recognize these unique patterns, making it more effective in protecting Filipinos from fraud, phishing attempts, and financial loss.

Is there a spam detection/prevention currently implemented in the Philippines?	Yes, there are spam detection and prevention measures currently implemented in the Philippines, mainly at the telecom level. Globe, Smart, and DITO have systems that block suspicious messages containing malicious keywords or links. For instance, Globe reported blocking over 2.7 billion scam and spam messages in 2022 following the directive of the National Telecommunications Commission (NTC). However, despite these measures, some personalized scam texts still reach subscribers, which shows that while prevention exists, it is not yet foolproof.
What should be the acceptable percentage of accuracy and F1-Score are?	For an SMS spam filter to be considered acceptable, its accuracy should ideally be at least 90% or higher. This ensures that the majority of spam messages are caught. However, accuracy alone is not enough; the F1-Score, which balances precision and recall, should also fall within 85–95%. This is important in the Philippine setting because blocking legitimate messages, such as local government emergency alerts, could have serious consequences, while failing to block fraudulent texts like fake GCash notifications could lead to financial harm. Both precision and recall need to be strong to provide reliable protection.
Do you think our current prototype is acceptable?	The current prototype can be considered acceptable if it already blocks the most common forms of scam texts in the Philippines, such as “Your GCash account is locked,” “Congratulations, you won a Toyota Vios,” or “Libre load, click this link.” As a proof of concept, it is a good start, but to be practical for everyday use, it should be trained on larger datasets that reflect local language use, including Taglish and regional dialects. Just like the filtering systems of Globe and Smart, the prototype should also be continuously updated since scammers regularly change their tactics. For research purposes, the prototype is acceptable, but for real-world deployment, further testing and improvements are necessary.

Date of Interview: September 9, 2025

Interviewee Name: Mr. Romnick Belen, Senior Software Engineer at Maya

Interviewee LinkedIn: <https://www.linkedin.com/in/romnick-belen-185622136>

Interviewer Name: Dominic Torres; Yoshida Al Zarouni, Aeron Valzado

Interview Video Drive Link:

https://lyceumofthephilippinesmy.sharepoint.com/:v/g/personal/2023-2-02911_lpunetwork_edu_ph/ETjuCj0HBvFPmh2n4LEJckBOg2rRnU7Pr8RXOdM37mciA?nav=eyJyZWZlcnJhbEluZm8iOmsicmVmZXJyYWxBcHAIiJPbmVEcm12ZUZvckJ1c2luZXNzliwicmVmZXJyYWxBcHBQbGF0Zm9ybSI6IldlYiIsInJlZmVycmFsTW9kZSI6InZpZXciLCJyZWZlcnJhbFZpZXciOiJNeUZpbGVzTGlua0NvcHkifX0&e=1nv3aZ

Question	Answer
In your opinion what are the impacts of the implementation of sim card registration in terms of spam?	SIM card registration has minimal impact on spam filtering. Its primary function is to link a SIM card to the user's identity, enabling authorities to trace and potentially disable SIM cards used for spam, rather than directly preventing spam messages.
What do you think are the key features of an SMS spam filter?	The key features of SMS spam involve differentiating unsolicited messages (spam) from legitimate ones (ham). An effective system should also enable users to validate classifications, ensuring that misidentified messages can be corrected and the accuracy of the spam filter is maintained.
How should we handle false positives and negatives in our model?	False positives in SMS spam filtering happen when legitimate messages are mistakenly marked as spam, which can cause users to miss important information. False negatives occur when actual spam messages pass through as normal, leading to frustration and security risks. To reduce these problems, the system should focus on improving accuracy by adjusting detection thresholds, learning from past errors, and using scoring methods that balance false positives and false negatives. Thresholds (e.g., moving from 0.5 to 0.7) can be tuned to improve performance, while weighting schemes can give higher penalties to dangerous spam, making the filter more reliable.

How would you ensure the privacy and security of SMS data in such a system?	To improve spam filtering accuracy, the system can apply penalties or weights to certain messages, helping reduce false positives and false negatives. At the same time, data handling must follow strict privacy principles: users should be consulted before collecting any information, only necessary data should be gathered, and all stored data must be protected securely. This ensures both effective spam detection and responsible data privacy management.
(Show functional and non-functional requirements)) Do you have any improvements or suggestions regarding our functional and non-functional requirements?	For functional requirements, the system should specify how data is managed, extracted, and sanitized, ensuring only necessary information is retained and properly processed. It should also include mechanisms for data accuracy evaluation, such as metrics to measure whether training data improves or reduces system performance. For non-functional requirements, improvements should emphasize performance (e.g., defining acceptable response times from the user's perspective), reliability (ensuring the system remains dependable even as spammers change tactics), and maintainability (documenting algorithms and code so updates can be applied without compromising functionality). Accuracy should also be highlighted as a critical aspect of both functional and non-functional requirements.
What advice would you give to student developers working on a project like this?	Student developers should ensure that the dataset used for training is properly cleaned and sanitized. Using unrefined or "dirty" data can significantly lower accuracy and waste effort, so data quality is just as important as the algorithms applied.